Intermediate Representations

If you are building a compiler from a high-level language to assembly language, you'll almost certainly need to take stop in the middle and create an intermediate representation of the program.

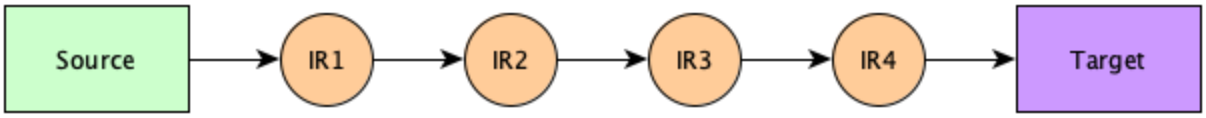
CONTENTS

Motivation • Flavors • Tuples • Control Flow Graphs • Single Static Assignment • Generating an IR • Real-life Examples of IRs • Summary

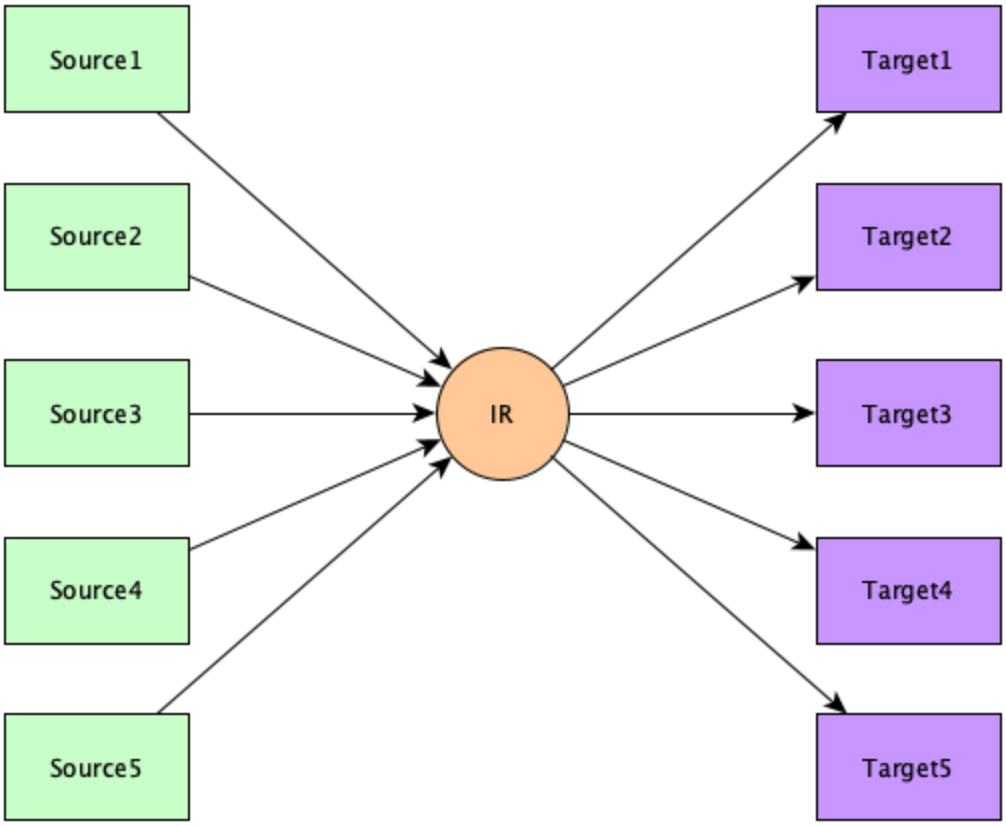
Motivation

An **intermediate representation** is any representation of a program "between" the source and target languages.

In a real compiler, you might see *several* intermediate representations!



A true *intermediate* representation is quite **independent** of the source and target languages, and yet very general, so that it can be used to build a whole family of compilers.



We use intermediate representations for at least four reasons:

1. Because translation appears to *inherently* require analysis and synthesis. Word-for-word translation does not work. We need a conceptual model of the program, one that we can easily create from the source code, and one that is easy to construct target code from.

2. To break the difficult problem of translation into two simpler, more manageable pieces.

3. To build retargetable compilers:
    - We can build new back ends for an existing front end (making the source language *more portable across machines)*.
    - We can build a new front-end for an existing back end (so a new machine can quickly get a set of compilers for different source languages).
    - We only have to write $2n$ half-compilers instead of $n(n-1)$ full compilers. (Though this might be a bit of an exaggeration in practice!)

4. To perform *machine independent* optimizations.

Flavors

Intermediate representations come in many flavors. The three main ones are:

Graph

Decorated and extended AST

Tuples

Code for an unbounded register machine

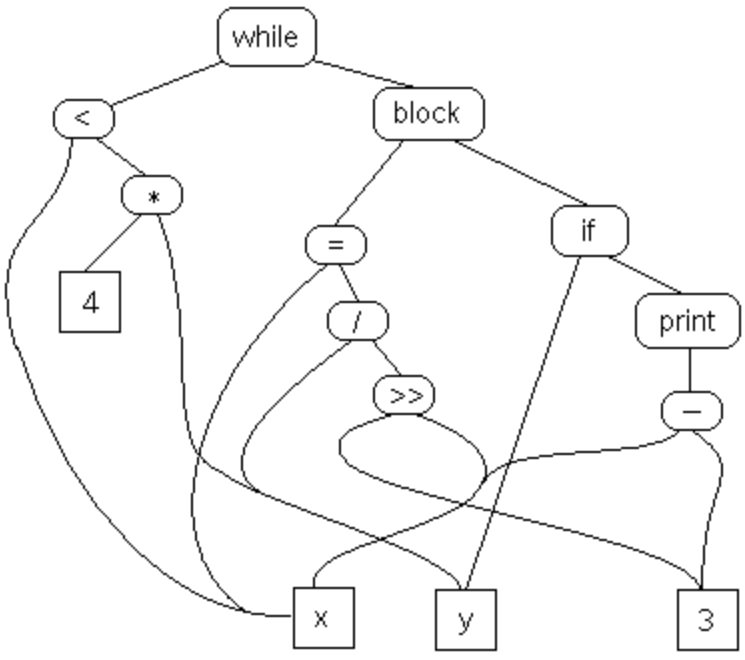
Stack

Code for a virtual stack machine

We'll use a running example code fragment to illustrate various options.

```
while (x < 4 * y) {
    x = y / 3 >> x;
    if (y) print x - 3;
}
```

The so-called **semantic graph** is is just an abstract syntax tree, analyzed, type checked, annotated and transformed into a graph:



Some might not call this an IR, but rather simply a **source program representation**: it's what the analyzer produces. You can even reconstruct the text of the source code from it (up to differences in whitespace and comments). Many folks would require a true IR to have some kind of lower level

instruction lists. **Tuples** are the most common. Here is the tuple representation of the example source code fragment above:

```
(JUMP, L2)          //     goto L2
(LABEL, L1)         // L1:
(SHR, 3, x, t0)     //     t0 := 3 >> x
(DIV, y, t0, t1)    //     t1 := y / t0
(COPY, t1, x)       //     x := t1
(JZ, y, L3)         //     if y == 0 goto L3
(SUB, x, 3, t2)     //     t2 := x - 3
(PRINT, t2)         //     print t2
(LABEL, L3)         // L3:
(LABEL, L2)         // L2:
(MUL, 4, y, t4)     //     t4 := 4 * y
(LT, x, t4, t5)     //     t5 < t4
(JNZ, t5, L1)       //     if t5 != 0 goto L1
```

Another common form for an IR is **stack code**. Here's how our running example might look:

```
    JUMP L2
L1:
    PUSH y
    PUSHC 3
    PUSH x
    SHR
    DIV
    STORE x
    PUSH y
    JZERO L3
    PUSH x
    PUSHC 3
    SUB
    PRINT
L3:
L2:
    PUSH x
    PUSHC 4
    PUSH y
    MUL
    LESS
    JNOTZERO L1
```

Each flavor of IR (graph, stack, tuples) processes **entities**. Each entity in the IR is a bundle of information, which will be used in the later phases of the compiler that generate real target code.

Entities are:

Entity	Properties	Notes
Literal	value	Integer or floating-point constant.
Variable	name type owner	Variable or parameter from the source code. The owner is the function or module in which the variable or parameter was declared.
Subroutine	name parent parameters locals	Procedure or Function.
Temporary	name type	A temporary variable, generated while creating the IR.
Label	name	A label used for jumps.

A type is i8, i16, i32, i64, i128, u8, u16, u32, u64, u128, f32, f64, f128, or similar. In an IR, all arguments are simple, meaning they all "fit" in a machine word. Arrays, strings, and structs will be represented by their base address alone (so its type is up to you, perhaps i64). Elements and fields will be accessed by their offset from the base. Also, there are no fancy types, so booleans, chars, and enums will be represented as integers. An IR should be source language agnostic and target language agnostic.

Tuples

The nice thing about intermediate representations is that, because they are intermediate, they can be whatever you want. You can literally make up a set of tuples!

Tuples are instruction-like objects consisting of an operator and zero or more operands (defined above). A typical set of tuples might include:

Tuple	Rendered as...	Description
`(COPY,x,y)`	`y := x`	Copy (a.k.a. Store)
`(ADD,x,y,z)`	`z := x + y`	Sum

`(SUB,x,y,z)`	`z := x - y`	Difference
`(MUL,x,y,z)`	`z := x * y`	Product
`(DIV,x,y,z)`	`z := x / y`	Quotient
`(MOD,x,y,z)`	`z := x mod y`	Modulo
`(REM,x,y,z)`	`z := x rem y`	Remainder
`(POWER,x,y,z)`	`z := pow x, y`	Exponentiation
`(SHL,x,y,z)`	`z := x << y`	Left shift
`(SHR,x,y,z)`	`z := x >> y`	Logical right shift
`(SAR,x,y,z)`	`z := x >>> y`	Arithmetic right shift
`(AND,x,y,z)`	`z := x & y`	Bitwise conjunction
`(OR,x,y,z)`	`z := x | y`	Bitwise disjunction
`(XOR,x,y,z)`	`z := x xor y`	Bitwise exclusive or
`(NOT,x,y)`	`y := !x`	Logical complement
`(NEG,x,y)`	`y := -x`	Negation
`(COMP,x,y)`	`y := ~x`	Bitwise complement
`(ABS,x,y)`	`y := abs x`	Absolute value
`(SIN,x,y)`	`y := sin x`	Sine
`(COS,x,y)`	`y := cos x`	Cosine
`(ATAN,x,y,z)`	`z := atan x,y`	Arctangent
`(LN,x,y)`	`y := ln x`	Natural logarithm
`(SQRT,x,y)`	`y := sqrt x`	Square root
`(INC x)`	`inc x`	Increment by 1; same as `(ADD,x,1,x)`
`(DEC,x)`	`dec x`	Decrement by 1; same as `(SUB,x,1,x)`
`(LT,x,y,z)`	`z := x < y`	1 if x is less than y; 0 otherwise
`(LE,x,y,z)`	`z := x <= y`	1 if x is less than or equal to y; 0 otherwise
`(EQ,x,y,z)`	`z := x == y`	1 if x is equal to y; 0 otherwise

`(NE,x,y,z)`	`z := x != y`	1 if x is not equal to y; 0 otherwise
`(GE,x,y,z)`	`z := x >= y`	1 if x is greater than or equal to y; 0 otherwise
`(GT,x,y,z)`	`z := x > y`	1 if x is greater than y; 0 otherwise
`(MEM_ADDR,x,y)`	`y := &x`	Copy address of x into y
`(MEM_GET,x,y)`	`y := *x`	Copy contents of memory at address x into y
`(MEM_SET,x,y)`	`*y := x`	Copy x into the memory at address y
`(MEM_INC x)`	`inc *x`	Increment a memory location given its address
`(MEM_DEC,x)`	`dec *x`	Decrement a memory location given its address
`(ELEM_ADDR,a,i,x)`	`x := &(a[i])`	Copy address of a[i] into z; identical to `(ADD, a, i*elsize, x)` where elsize is the size in bytes of the element type of array a.
`(ELEM_GET,a,i,x)`	`x := a[i]`	Copy contents of memory at address a + i*elsize into x; where elsize is the size in bytes of the element type of array a
`(ELEM_SET,a,i,x)`	`a[i] := x`	Copy x into the memory at address a + i*elsize; where elsize is the size in bytes of the element type of array a
`(FIELD_ADDR,s,f,x)`	`x := &(s.f)`	Copy address of s.f into x; identical to `(ADD, s, ofs(f), x)` where ofs(f) is the byte offset of field f in struct s.
`(FIELD_GET,s,f,x)`	`x := s.f`	Copy contents of memory at address s + ofs(f)

`(FIELD_SET,s,f,x)`	`s.f := x`	Copy x into the memory at address s + ofs(f)
`(LABEL,L)`	`L:`	Label
`(JUMP,L)`	`goto L`	Unconditional jump to a label
`(JZERO,x,L)`	`if x == 0 goto L`	Jump if zero / Jump if false
`(JNZERO,x,L)`	`if x != 0 goto L`	Jump if zero / Jump if true
`(JLT,x,y,L)`	`if x < y goto L`	Jump if less / Jump if not greater or equal
`(JLE,x,y,L)`	`if x <= y goto L`	Jump if less or equal / Jump if not greater
`(JEQ,x,y,L)`	`if x == y goto L`	Jump if equal
`(JNE,x,y,L)`	`if x != y goto L`	Jump if not equal
`(JGE,x,y,L)`	`if x >= y goto L`	Jump if greater or equal / Jump if not less
`(JGT,x,y,L)`	`if x > y goto L`	Jump if greater / Jump if not less
`(MULADD,x,y,z,w)`	`w := x + y*z`	Multiply then add
`(IJ,x,dx,L)`	`x := x + dx; goto L`	Increment x then unconditionally jump
`(IJE,x,dx,y,L)`	`x += dx; if x == y goto L`	Increment and jump if equal (useful at end of for-loops that count up to a given value)
`(DJNZ,x,L)`	`if --x != 0 goto L`	Decrement and jump if not zero (great for loops that count down to zero by one)
`(CALLP,f,x1,...,xn)`	`call f, x1, ..., xn`	Call procedure (non-value returning function) f, with arguments x1, ..., xn
`(CALLF,f,x1,...,xn,x)`	`x := call f, x1, ..., xn`	Call function f, with arguments x1, ..., xn, and copy the result into x
`(RETP)`	`ret`	Return from procedure
`(RETF,x)`	`ret x`	Return x from this function

`(INT_TO_STR,x,y)`	`y := to_string x`	String representation of an integer (one can add a new tuple to take a base)
`(FLOAT_TO_STR,x,y)`	`y := to_string x`	String representation of a float (one can add a new tuple taking a format string)
`(BOOL_TO_STR,x,y)`	`y := to_string x`	String representation of a boolean (localized?)
`(CHAR_TO_STR,x,y)`	`y := to_string x`	String representation of a character
`(ALLOC,x,y)`	`y := alloc x`	Allocate x bytes of memory, returning the address in y (or 0 if the memory could not be allocated).
`(ARRAY_ALLOC,x,y)`	`y := array_alloc x`	Allocate x*elsize bytes of memory, where elsize is the number of bytes of an element in array x, returning the address in y (or 0 if the memory could not be allocated).
`(DEALLOC,x)`	`dealloc x`	Deallocate the memory at address x (or do nothing if x is 0)
`(INT_TO_FLOAT,x,y)`	`y := to_float x`	Integer to float
`(ASSERT_NOT_NULL,x)` `(ASSERT_NONZERO,x)`	`assert x != 0`	Do something if x is null (details left to the implementation)
`(ASSERT_POSITIVE,x)`	`assert x > 0`	Do something if x is not positive (details left to the implementation)
`(ASSERT_BOUND,x,y,z)`	`assert y <= x < z`	Do something if x is not between y (inclusive) and x (inclusive) (details left to the implementation)

`(NO_OP)`	`nop`	Do nothing
`(EXIT,x)`	`exit`	Terminate the program with error code x

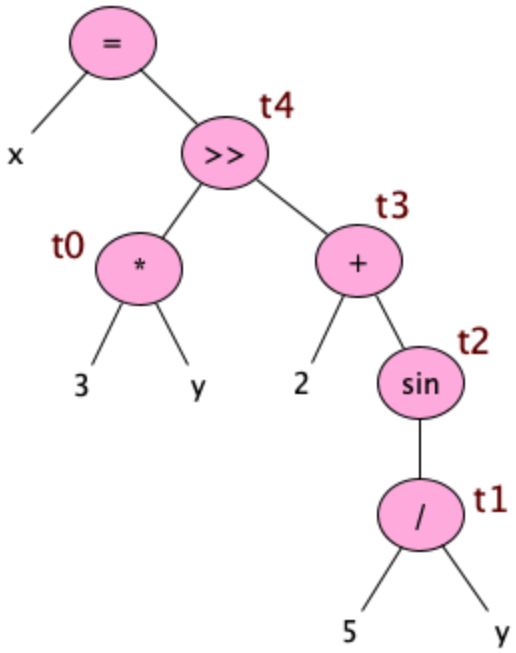
Arithmetic

Note how tuples assume what is essentially an unlimited supply of registers. These are often called **temporaries**. Temporaries differ from variables in that variables come from source language variables that are actually declared, while temporaries are the result of evaluating expressions. Here's an example:

```
x = 3 * y >> 2 + sin(5 / y);
```

```
(MUL, 3, y, t0)
(DIV, 5, y, t1)
(SIN, t1, t2)
(ADD, 2, t2, t3)
(SHR, t0, t3, t4)
(COPY, t4, x)
```

Note how the temporaries are assigned while walking the AST, in a post-order-ish sort of manner:



If Statements

For a typical if-else, generate labels and jumps:

```
if x > 0 {     (JGT, x, 0, L1)
  print y;     (PRINT, x)
} else {       (JUMP, L2)
  print y;     (LABEL, L1)
}              (PRINT, y)
               (LABEL, L2)
```

While Loops

Here is a while loop. We always generate labels at the beginning (for jumping back to the top after a loop body is complete, and also as the target of a `continue` statement) and the end (in order to escape via a natural end or a `break`).

```
while y > 3 {     (LABEL, L1)       // always mark top
  print y;        (JLE, y, 3, L2)
  y--;            (PRINT, y)
  break;          (DEC, y)
  x++;            (JUMP, L2)        // break means jump to end
  continue;       (INC, x)
  x = 1;          (JUMP, L1)        // continue means jump to top
}                 (COPY, 1, x)
                  (JUMP, L1)        // end of while, jump to top
                  (LABEL, L2)       // always mark end
```

Short-Circuit Logic

TODO

Strings

TODO

Arrays

We can make use of relatively powerful tuples to handle arrays:

```
a[2] = 5;          (ELEM_SET, a, 2, 5)
x = a[y * 3]       (MUL, y, 3, t0)
                   (ELEM_GET, a, t0, x)
```

If your arrays are multi-dimensional, we need to find the base-address of a row. This is where `ELEM_ADDR` comes in:

```
print a[i][j][k]     (ELEM_ADDR, a, i, t0)
                     (ELEM_ADDR, t0, j, t1)
                     (ELEM_GET, t1, k, t2)
                     (PRINT,t2)
```

Here's an example with a for-loop:



```
for (int i = 0; i < n; i += di)
    a[i][j+2] = j;
```

```
(COPY, 0, i)            //      i := 0
(LABEL, L1)             // L1:
(JGE, i, n, L2)         //      if i >= n goto L2
(ELEM_ADDR, a, i, t0)   //      t0 := &a[i]
(ADD, j, 2, t1)         //      t1 := j + 2
(ELEM_SET, j, t0, t1)   //      t0[t1] := j
(IJ, i, di, L1)         //      i += di, goto L1
(LABEL, L2)             // L2:
```

Structs

A simple example first:

```
struct Point {      (STRUCT_ALLOC, Point, p)
  int x; int y;     (FIELD_SET, p, x, 3)
}                   (FIELD_SET, p, y, 4)
let p: Point;       (FIELD_GET, p, y, t0)
p.x = 3;            (PRINT, t0)
p.y = 4;
print p.y;
```

For nested structs, FIELD_ADDR is helpful:

TODO

For fun, here is an example with arrays and structs:

```
let a: int[];                    (DATA, 32, 16, 8, 4, 2, 1, a)
let a = [32, 16, 8, 4, 2, 1];    (ELEM_SET, 21, a, 3)
a[3] = 21;                       (ELEM_GET, a, 5, t0)
let x = a[a[5]];                 (ELEM_GET, a, t0, x)
struct Point {                   (ARRAY_ALLOC, 50, polygon)
  int x; int y;                  (ELEM_ADDR, polygon, 5, t2)
}                                (FIELD_GET, t2, y, t3)
let polygon: Point[50];          (PRINT, t3)
print(polygon[5].y);
```

Subroutines

Let's start with a simple procedure call:

```
send(x+5, y, "hello");     (ADD, x, 5, t0)
                           (CALLP, send, t0, y, s0)
                           .
                           .
                           .
                           s0: [104, 101, 108, 108, 111, 0]
```

Now here is a definition and a call:

```
func f(i: float): string {
  return "dog" unless i >= 3.3;
}
let test: string = f(cos(2.2));
```

```
main:
  (COS, 2.2, t0)
  (CALLF, f, t0, t1)
  (COPY, t1, test)
  (EXIT)
f:
  (JGE, i, 3.3, L0)
  (RET, s0)
L0:
  (RET)
s0:
  [100, 111, 103]
```

Here's an example with recursion:

TODO

Threads

```
func f() {
    for i in 0..<20000 {
        print("\(i) ");
    }
}
let c1: thread = start f();
let c2: thread = start f();
join(c1);
join(c2);
```

```
main:
  (CALL, __create_thread, f, c1)
  (CALL, __create_thread, f, c2)
  (CALL, __join, c1)
  (CALL, __join, c2)
  (EXIT)
f:
  (COPY, 0, i)
L0:
  (JGE, i, 20000, L1)
  (INT_TO_STRING, i, t0)
  (CALL, __concat_string, s0, t0, t1)
  (PRINT, t1)
  (INC_JUMP, i, L0)
L1:
  (RET)
s0:
  [32, 0]
```

Lower-level Tuples

The tuples we described so far have been quite powerful. They know whether their operands are floats or ints and how big they are. They know the sizes of arrays and structs. They know how much space has been allocated for arrays and automatically do bounds checking if needed.

Eventually, all of this size information will have to appear in machine code. So a good question is where do we start making it explicit. We could take care of all this when we generate assembly language code, or, we can do one more level of IR code generating, using lower level tuples. For example, in this C++ fragment:

```
double a[20][10];

// ...

for (int i = 0; i < n; i += di)
    a[i][j+2] = j;
```

we have an array of doubles with 20 rows and 10 columns. Doubles are 8 bytes each, so we know each row is 80 bytes. So we know the address of a[0] is the same as the address of a, a[1] is at a+80, a[2] at a+160, and so on. We can do all of the address computations ourselves, avoiding all of the fancy IR tuples, and using only MEM_GET and MEM_SET:

```
(COPY, 0, i)            //       i := 0
(LABEL, L1)             //  L1:
(JGE, i, n, L2)         //       if i >= n goto L2
(MUL, i, 80, t0)        //       t0 := i * 80
(ADD, a, t0, t1)        //       t1 := a + t0
(ADD, j, 2, t2)         //       t2 := j + 2
(MUL, t2, 8, t3)        //       t3 := t2 * 8
(ADD, t1, t3, t4)       //       t4 := t1 + t3
(MEM_SET, j, t4)        //       *t4 := j
(ADD, i, di, i)         //       i := i + di
(JUMP, L1)              //       goto L1
(LABEL, L2)             //  L2:
```

TODO - bounds checking? allocation?

Control Flow Graphs

Now that we've seen the basics of tuples, let's see how they are stored and manipulated by a compiler.

A **control flow graph** is a graph whose nodes are basic blocks and whose edges are transitions between blocks.

A **basic block** is a:

- maximal-length sequence of instructions that will execute in its entirety
- maximal-length straight-line code block
- maximal-length code block with only one entry and one exit

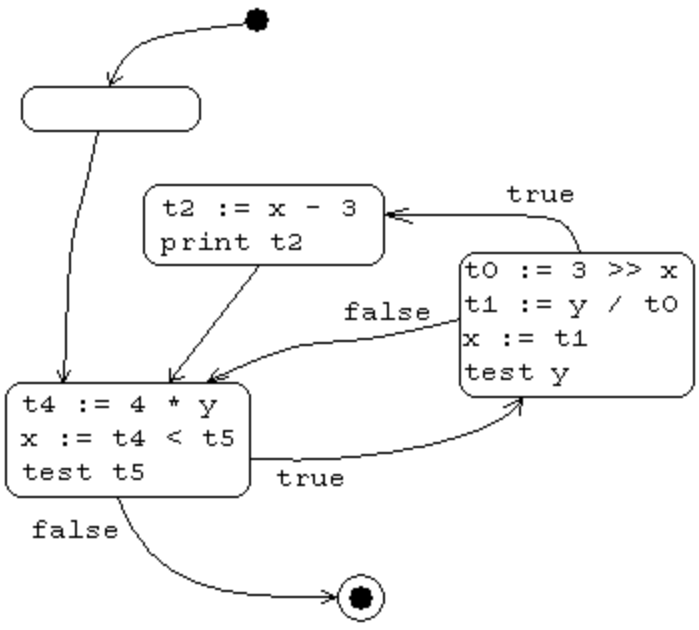
... in the abscence of hardware faults, interrupts, crashes, threading problems, etc.

To locate basic blocks in flattened code:

- **Starts with**: (1) target of a branch (label) or (2) the instruction after a conditional branch
- **Ends with**: (1) a branch or (2) the instruction before the target of a branch.

Here's an example:

```
     goto L2
L1:
     t0 := 3 >> x
     t1 := y / t0
     x := t1
     if y == 0 goto L3
     t2 := x - 3
     print t2
L3:
L2:
     t4 := 4 * y
     x := t4 < t5
     if t5 != 0 goto L1
```



Single Static Assignment

No notes here yet, for now, see Wikipedia.

Generating an IR

In a compiler, you often emit the "first" IR by navigating the CST or the AST and outputting a semantic graph. The graph is a semantically analyzed (decorated) version of the AST. It's not necessarily a tree because each of the basic entities (variables, functions, types, etc.) may be referenced multiple times.

> ***Writing a Transpiler?***
>
> *If so, you will likely just generate target code directly from the semantic graph. Then you're done and you don't have to read on.*
>
> *But if you are looking to make a more traditional compiler, with machine code as your ultimate goal, read on.*

The next level of the IR will be either tuple-based or stack based. You'll be grouping the tuples into basic blocks for further analysis, but first let's look at really simple example. To get the idea, here's a quick-and-dirty illustration of a Haskell program that navigates a semantic graph in a tiny language and outputs an instruction list:

compiler1.hs

```
--  A little Compiler from a trivial imperative language to a small
--  single-accumulator machine.

--  Source Language
--
--  Exp  =  numlit  |  id  |  Exp  "+"  id
--  Bex  =  Exp  "="  id  |  "not"  Bex
--  Com  =  "skip"
--       |   id  ":="  Exp
--       |   Com  ";"  Com
--       |   "while"  Bex  "do"  Com  "end"

data Exp                    -- Arithmetic Expressions can be
```

```
    = Lit Int             --   literals
    | Ident String        --   identifiers
    | Plus Exp String     --   binary plus expressions, left associative

data Bex                  -- Boolean Expressions can be
    = Eq Exp String       --   equality comparisons btw exp and identifiers
    | Not Bex             --   unary not applied to boolean expressions

data Com                  -- Commands can be
    = Skip                --   skips (no-ops)
    | Assn String Exp     --   assignment of an arithmetic expr to an id
    | Seq [Com]           --   a sequence (list) of commands
    | While Bex Com       --   a while-command with a boolean condition

--  Target Language
--
--  The target language is an assembly language for a machine with a
--  single accumulator and eight simple instructions.

data Inst
    = LDI Int             -- Load Immediate
    | LD String           -- Load
    | ST String           -- Store
    | NOT                 -- Not
    | EQL String          -- Equal
    | ADD String          -- Add
    | JMP Int             -- Jump
    | JZR Int             -- Jump if zero
    deriving (Show)

--  The Compiler
--
--  ke e loc   the code from compiling expression e at address loc
--  kb b loc   the code from compiling boolexp b at address loc
--  kc c loc   the code from compiling command c at address loc

ke :: Exp -> Int -> [Inst]
ke (Lit n) loc = [LDI n]
ke (Ident x) loc = [LD x]
```

```
ke (Plus e x) loc = ke e loc ++ [ADD x]

kb :: Bex -> Int -> [Inst]
kb (Eq e x) loc = ke e loc ++ [EQL x]
kb (Not b) loc = kb b loc ++ [NOT]

kc :: Com -> Int -> [Inst]
kc (Skip) loc = []
kc (Assn x e) loc = ke e loc ++ [ST x]
kc (Seq []) loc = []
kc (Seq (c:cs)) loc =
    let f = kc c loc in f ++ kc (Seq cs) (loc + length f)
kc (While b c) loc =
    let f1 = kb b loc in
        let f2 = kc c (loc + length f1 + 1) in
            f1 ++ [JZR (loc + length f1 + 1 + length f2 + 1)] ++ f2 ++ [JMP loc]

--  Pretty-Printing a target program

pprint [] loc = putStrLn ""
pprint (h:t) loc = printInst h loc >> pprint t (loc + 1)
    where printInst h loc = putStrLn (show loc ++ "\t" ++ show h)

--  An Example Program
--
--  x := 5;
--  skip;
--  while not 8 = y do
--    z := (19 + x) + y;
--  end;
--  x := z;

prog1 =
  Seq [
    (Assn "x" (Lit 5)),
    Skip,
    (While
      (Not (Eq (Lit 8) "y"))
      (Assn "z" (Plus (Plus (Lit 19) "x") "y"))),
```

```
    (Assn "x" (Ident "z"))]

-- Top-level call to invoke the Compiler
compile c loc = pprint (kc c loc) loc

-- Since this is just a prototype, run this file as a script
main = compile prog1 0
```

Here's the output:

```
$ runhaskell compiler1.hs
0       LDI 5
1       ST "x"
2       LDI 8
3       EQL "y"
4       NOT
5       JZR 11
6       LDI 19
7       ADD "x"
8       ADD "y"
9       ST "z"
10      JMP 2
11      LD "z"
12      ST "x"
```

Real-life Examples of IRs

Here are a few things that qualify as intermediate representations, or, at least, things a compiler front-end may output:

GNU RTL

The intermediate language for the many source and target languages of the GNU Compiler Collection.

PCODE

The intermediate language of early Pascal compilers. Stack based. Responsible for wide adoption of Pascal in the 1970s.

Java Virtual Machine

Another virtual machine specification. Almost all Java compilers use this format. So do nearly all Scala, Ceylon, Kotlin, and Groovy compilers. Hundreds of other languages use it as well. JVM code can be interpreted, run on specialized hardware, or jitted.

Rust Intermediate Languages

The compiler Rustc transforms source first in to a High-Level Intermediate Language (HIR), then into the Mid-Level Intermediate Language (MIR), then into LLVM. The MIR was introduced in this blog post. You can read more in the original request for it and in the Rustc book.

CIL

Common Intermediate Language. Languages in Microsoft's .NET framework (such as C+, VB.NET, etc.) compile to CIL, which is then assembled into bytecode.

C

Why not? It's widely available and the whole back end is already done within the C compiler.

C--

Kind of like using C, but C-- is designed explicitly to be an intermediate language, and even includes a run-time interface to make it easier to do garbage collection and exception handling. Seems to be defunct.

LLVM

The new hotness. Much more than just a VM.

SIL

The Swift Intermediate Language. Here is a nice presentation on SIL.

asm.js

A low-level subset of JavaScript.

Web Assembly

An efficient and fast stack-based virtual machine.

A JVM Example

TODO

An LLVM Example

TODO

An SIL Example

TODO

A Web Assembly Example

A C++ function:



```
unsigned gcd(unsigned x, unsigned y) {
  while (x > 0) {
    unsigned temp = x;
    x = y % x;
    y = temp;
  }
  return y;
}
```

Compiled to Web Assembly:

```
(module
  (type $type0 (func (param i32 i32) (result i32)))
  (table 0 anyfunc)
  (memory 1)
  (export "memory" memory)
  (export "_Z3gcdjj" $func0)
  (func $func0 (param $var0 i32) (param $var1 i32) (result i32)
    (local $var2 i32)
    block $label0
      get_local $var0
      i32.eqz
      br_if $label0
      loop $label1
        get_local $var1
        get_local $var0
        tee_local $var2
        i32.rem_u
        set_local $var0
        get_local $var2
        set_local $var1
        get_local $var0
        br_if $label1
      end $label1
      get_local $var2
      return
    end $label0
```

```
      get_local $var1
    )
  )
```

Case Study: An IR for C

Guess what? C is such a simple language, an IR is fairly easy to design. Why is C so simple?

- Only base types are several varieties of ints and floats.
- No true arrays — `e1[e2]` just abbreviates `*(e1+e2)`; *all* indicies start at zero, there's no bounds checking.
- All parameters passed by value, and in order.
- Block structure is very restricted: all functions on same level (no nesting); variables are either truly global or local to a top-level function; implementation drastically simplified because no static links are ever needed and functions can be easily passed as parameters without scoping trouble.
- Structure copy is bitwise.
- Arrays aren't copied element by element: array assignment is just an assignment of the starting address.
- Language is small; nearly everything interesting (like I/O) is in a library. So we only need tuples for the core language.

The tuples already given are more than sufficient to use in a C compiler.

Summary

We've covered:

✅ ...

✅ ...

x86 Architecture Overview

The IA-32 is the instruction set architecture (ISA) of Intel's most successful line of 32-bit processors, and the Intel 64 ISA is its extension into 64-bit processors. (Actually Intel 64 was invented by AMD, who called it x86-64). These notes summarize a few items of interest about these two ISAs. They in no way serve as a substitute for reading Intel's manuals.

CONTENTS

IA-32 and x86-64 • x86-64 Architecture Diagram • Registers • Instruction Set • Addressing Memory • Flags Register • The System Developer's Manual

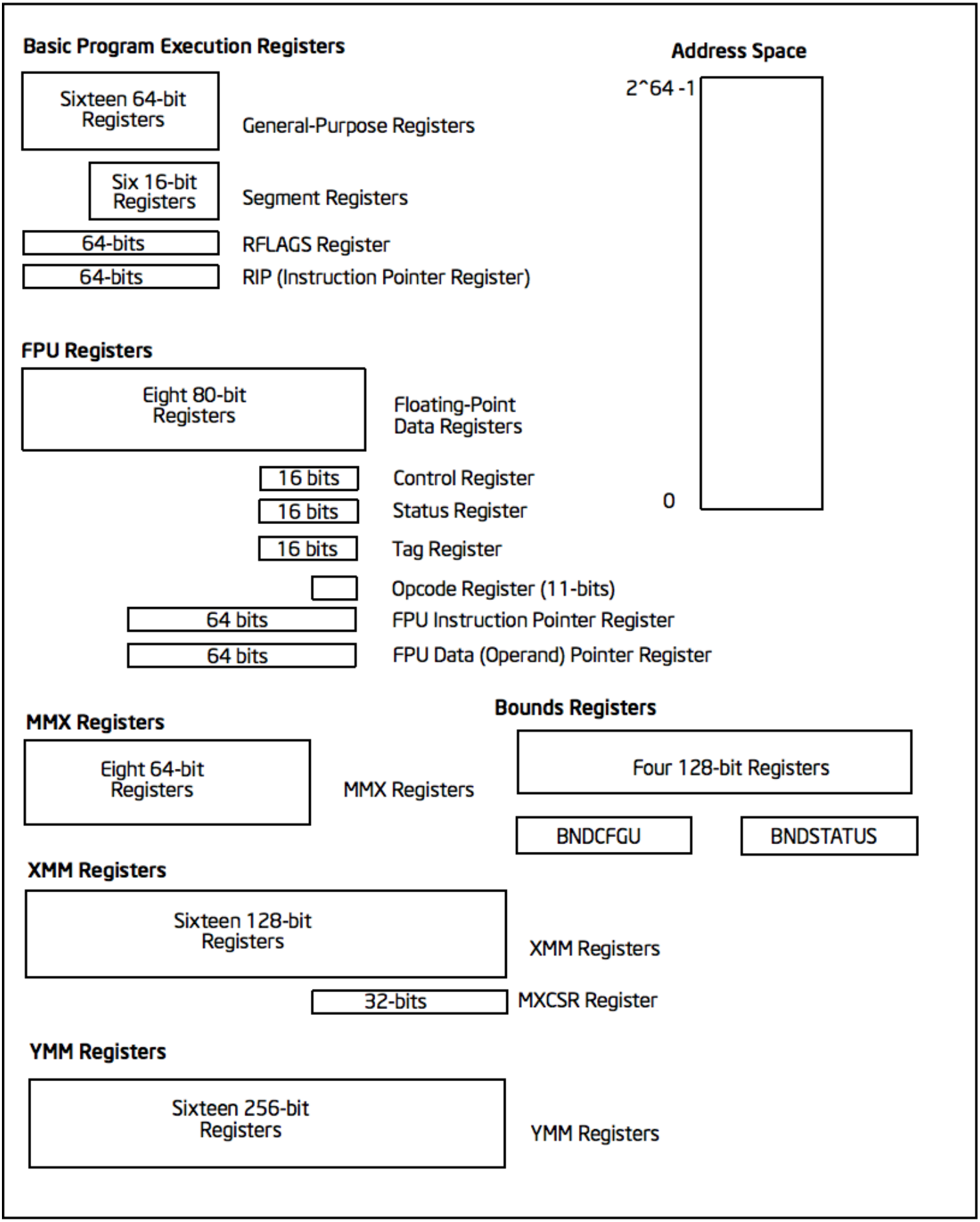
IA-32 and x86-64

The two massively popular architectures IA-32 and x86-84 are so common, they are described in a single set of manuals.

The following notes briefly summarize the latter architecture only.

x86-64 Architecture Diagram

The basic architecture of the x86-64 is described in Volume 1 of the System Developer's Manual. The following diagram is taken directly from Chapter 3 in this volume:


Basic Program Execution Registers
Sixteen 64-bit Registers
General-Purpose Registers
Six 16-bit Registers
Segment Registers
64-bits
RFLAGS Register
64-bits
RIP (Instruction Pointer Register)
Address Space
2^64 -1
0
FPU Registers
Eight 80-bit Registers
Floating-Point Data Registers
16 bits
Control Register
16 bits
Status Register
16 bits
Tag Register
Opcode Register (11-bits)
64 bits
FPU Instruction Pointer Register
64 bits
FPU Data (Operand) Pointer Register
MMX Registers
Eight 64-bit Registers
MMX Registers
Bounds Registers
Four 128-bit Registers
BNDCFGU
BNDSTATUS
XMM Registers
Sixteen 128-bit Registers
XMM Registers
32-bits
MXCSR Register
YMM Registers
Sixteen 256-bit Registers
YMM Registers

Registers

Application Programmers generally use only the general purpose registers, floating point registers, XMM, and YMM registers.

General Purpose Registers

These are 64 bits wide and used for integer arithmetic and logic, and to hold both data and pointers to memory. The registers are called R0...R15. Also:

- You can access the lower order 32-bits of each register using the names R0D...R15D. The "D" stands for "doubleword" because strangely, the word "word" on this platform refers to a 16-bit quantity. Why? Backward compatibilty! The x86-64 grew out of a 16-bit processor family created in the 1970s.
- You can access the lower order 16-bits of each register using the names R0W...R15W.
- You can access the lower order 8-bits of each register using the names R0B...R15B.
- R0...R7 have aliases RAX, RCX, RBX, RDX, RSP, RBP, RSI, RDI, respectively.
- R0D...R7D have aliases EAX, ECX, EBX, EDX, ESP, EBP, ESI, EDI, respectively.
- R0W...R7W have aliases AX, CX, BX, DX, SP, BP, SI, DI, respectively.
- R0B...R7B have aliases AL, CL, BL, DL, SPL, BPL, SIL, DIL, respectively.

RIP and RFLAGS

RIP is the instruction pointer and RFAGS is the flags register.

Segment Registers

These are CS, DS, SS, ES, FS, and GS. I haven't used them in 64-bit programming.

XMM Registers

These are 128-bits wide. They are named XMM0...XMM15. Use them for floating-point and integer arithmetic. You can do operations on 128-bit integers, but you can also take advantage of their ability

to do operations in parallel:

- Two 64-bit integer operations in parallel
- Four 32-bit integer operations in parallel
- Eight 16-bit integer operations in parallel
- Sixteen 8-bit integer operations in parallel
- Two 64-bit floating-point operations in parallel
- Four 32-bit floating-point operations in parallel

YMM Registers

These are 256-bits wide. They are named YMM0...YMM15. Use them for floating-point arithmetic. You can do:

- Four 64-bit floating-point operations in parallel
- Eight 32-bit floating-point operations in parallel

and some other crazy things.

FPU Registers

There are eight registers used for computing with 80-bit floating point values. The registers don't have names because they are used in a stack-like fashion.

Other Registers

Application programmers can remain oblivious of the rest of the registers:

- The 8 32-bit **processor control registers**: CR0, CR1, CR2, CR3, CR4, CR5, CR6, CR7. The lower 16 bits of CR0 is called the Machine Status Word (MSW).
- The 4 16-bit **table registers**: GDTR, IDTR, LDTR and TR.
- The 8 32-bit **debug registers**: DR0, DR1, DR2, DR3, DR4, DR5, DR6 and DR7.
- The 5 test registers: TR3, TR4, TR5, TR6 and TR7.
- The **memory type range registers**
- The **machine specific registers**
- The **machine check registers**

Instruction Set

See the SDM Volume 1, Chapter 5 for a nice overview of **all** of the processor instructions and Volume 2 for complete information.

The following table shows most of the available instructions, using the instruction names as specified in the Intel syntax. Not every processor supports every instruction, of course.

The vertical bar means OR, the square brackets mean OPTIONAL, and parentheses are used for grouping. For example:

- `SH(L|R)[D]` stands for `SHL`, `SHR`, `SHLD`, `SHRD`.
- `PUSH[A[D]]` stands for `PUSH`, `PUSHA`, `PUSHAD`.

INTEGER	FPU	SSE	SSE2
MOV CMOV[N]((L|G|A|B)[E]|E|Z|S|C|O|P) XCHG BSWAP XADD CMPXCHG[8B] PUSH[A[D]] | POP[A[D]] IN | OUT CBW | CWDE | CWD | CDQ MOVSX | MOVZX ADD | ADC SUB | SBB [I]MUL [I]DIV INC | DEC NEG CMP DAA | DAS AAA | AAS | AAM | AAD AND | OR | XOR | NOT SH(L|R)[D] SA(L|R) RO(L|R) RC(L|R) BT[S|R|C] BS(F|R) SET[N]((L|G|A|B)[E]|E|Z|S|C|O|P) TEST JMP J[N]((L|G|A|B)[E]|E|Z|S|C|O|P) J[E]CXZ LOOP[N][Z|E] CALL | RET INT[O] | IRET ENTER | LEAVE BOUND MOVS[B|W|D] CMPS[B|W|D]	F[I]LD F[I]ST[P] FBLD FBSTP FXCH FCMOV[N](E|B|BE|U) FADD[P] FIADD FSUB[R][P] FISUB[R] FMUL[P] FIMUL FDIV[R][P] FIDIV[R] FPREM[1] FABS FCHS FRNDINT FSCALE FSQRT FXTRACT F[U]COM[P][P] FICOM[P] F[U]COMI[P] FTST FXAM FSIN FCOS FSINCOS FPTAN FPATAN F2XM1 FYL2X FYL2XP1 FLD1 FLDZ FLDPI FLDL2E FLDLN2 FLDL2T FLDLG2	MOV(A|U)PS MOV(H|HL|L|LH)PS MOVSS MOVMSKPS ADD(P|S)S SUB(P|S)S MUL(P|S)S DIV(P|S)S RCP(P|S)S SQRT(P|S)S RSQRT(P|S)S MAX(P|S)S MIN(P|S)S CMP(P|S)S [U]COMISS ANDPS ANDNPS ORPS XORPS SHUFPS UNPCK(H|L)PS CVTPI2PS CVT[T]PS2PI CVTSI2SS CVT[T]SS2SI PAVG(B|W) PEXTRW PINSRW P(MIN|MAX)(UB|SW) PMOVMSKB PMULHUW PSADBW PSHUFW LDMXCSR STMXCSR MASKMOVQ MOVNT(Q|PS)	MOV(A|U)PD MOV(H|L)PD MOVSD MOVMSKPD ADD(P|S)D SUB(P|S)D MUL(P|S)D DIV(P|S)D SQRT(P|S)D MAX(P|S)D MIN(P|S)D CMP(P|S)D [U]COMISD ANDPD ANDNPD ORPD XORPD SHUFPD UNPCK(H|L)PD CVT(PI|DQ)2PD CVT[T]PD2(PI|DQ) CVTSI2SD CVT[T]SD2SI CVTPS2PD CVTPD2PS CVTDQ2PS CVT[T]PS2DQ CVTSS2SD CVTSD2SS MOVDQ(A|U) MOVQ2DQ MOVDQ2Q PUNPCK(H|L)QDQ PADDQ PSUBQ PMULUDQ PSHUF(LW|HW|D) PS(L|R)LDQ

SCAS[B|W|D]
LODS[B|W|D]
STOS[B|W|D]
INS[B|W|D]
OUTS[B|W|D]
REP[N][Z|E]

STC | CLC | CMC
STD | CLD
STI | CLI
LAHF | SAHF
PUSHF[D] | POPF[D]

LDS | LES | LFS | LGS | LSS

LEA
NOP
UD2
XLAT[B]
CPUID

FINCSTP
FDECSTP
FFREE
F[N]INIT
F[N]CLEX
F[N]STCW
FLDCW
F[N]STENV
FLDENV
F[N]SAVE
FRSTOR
F[N]STSW
FWAIT | WAIT
FNOP

FXSAVE
FXRSTOR

PREFETCHT(0|1|2)
PREFETCHNTA
SFENCE

MASKMOVDQU
MOVNT(PD|DQ|I)
CLFLUSH
LFENCE
MFENCE
PAUSE

SYSTEM	MMX	SSE3	SSE4
LGDT | SGDT LLDT | SLDT LTR | STR LIDT | SIDT LMSW | SMSW CLTS ARPL LAR LSL VERR | VERW INVD | WBINVD INVLPG LOCK HLT RSM RDMSR | WRMSR RDPMC RDTSC SYSENTER SYSEXIT	MOVD MOVQ PACKSS(WB|DW) PACKUSWB PUNPCK(H|L)(BW|WD|DQ) PADD(B|W|D) PADD(S|US)(B|W) PSUB(B|W|D) PSUB(S|US)(B|W) PMUL(H|L)W PMADDWD PCMP(EQ|GT)(B|W|D) PAND PANDN POR PXOR PS(L|R)L(W|D|Q) PSRA(W|D) EMMS	FISTTP LDDQU ADDSUBP(S|D) HADDP(S|D) HSUBP(S|D) MOVS(H|L)DUP MOVDDUP MONITOR MWAIT	PMUL(LD|DQ) DPP(D|S) MOVNTDQA BLEND[V](PD|PS) PBLEND(VB|W) PMIN(UW|UD|SB|SD) PMAX(UW|UD|SB|SD) ROUND(P|S)(S|D) EXTRACTPS INSERTPS PINSR(B|D|Q) PEXTR(B|W|D|Q) PMOV(S|Z)X(BW|BD|WD|BQ|WQ|DQ) MPSADBW PHMINPOSUW PTEST PCMPEQQ PACKUSDW PCMP(E|I)STR(I|M) PCMPGTQ CRC32 POPCNT

64-BIT MODE	VIRTUAL MACHINE	SSSE3	AESNI
CDQE CMPSQ CMPXCHG16B LODSQ MOVSQ MOVZX STOSQ SWAPGS SYSCALL SYSRET	VMPTRLD VPTRST VMCLEAR VMREAD VMWRITE VMCALL VMLAUNCH VMRESUME VMXOFF VMXON INVEPT INVVPID	PHADD(W|SW|D) PHSUB(W|SW|D) PABS(B|W|D) PMADDUBSW PMULHRSW PSHUFB PSIGN(B|W|D) PALIGNR	AESDEC[LAST] AESENC[LAST] AESIMC AESKEYGENASSIST PCLMULQDQ

Addressing Memory

In protected mode, applications can choose a flat or segmented memory model (see the SDM Volume 1, Chapter 3 for details); in real mode only a 16-bit segmented model is available. Most programmers will only use protected mode and a flat-memory model, so that's all we'll discuss here.

A memory reference has four parts and is often written as

```
[SELECTOR : BASE + INDEX * SCALE + OFFSET]
```

The selector is one of the six segment registers; the base is one of the eight general purpose registers; the index is any of the general purpose registers except ESP; the scale is 1, 2, 4, or 8; and the offset is any 32-bit number. (Example: `[fs:ecx+esi*8+93221]` .) The minimal reference consists of only a base register or only an offset; a scale can only appear if there is an index present.

Sometimes the memory reference is written like this:

```
selector
offset(base,index,scale)
```

Data Types

The data types are

Type name	Number of bits	Bit indices
Byte	8	7..0
Word	16	15..0
Doubleword	32	32..0
Quadword	64	63..0
Doublequadword	128	127..0

Little Endianness

The IA-32 is little endian, meaning the least significant bytes come first in memory. For example:

0	12
1	31
2	CB
3	74
4	67
5	45
6	0B
7	23
8	A4
9	1F
A	36
B	06
C	FE
D	7A
E	12

```
byte @ 9 = 1F
word @ B = FE06
word @ 6 = 230B
word @ 1 = CB31
dword @ A = 7AFE0636
qword @ 6 = 7AFE06361FA4230B
word @ 2 = 74CB
qword @ 3 = 361FA4230B456774
dword @ 9 = FE06361F
```

Note that if you draw memory with the lowest bytes at the bottom, then it is easier to read these values!

Flags Register

Many instructions cause the flags register to be updated. For example if you execute an `add` instruction and the sum is too big to fit into the destination register, the `Overflow` flag is set.

```
 3 3 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0
 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0 9 8 7 6 5 4 3 2 1 0
+---------------------------------------------------------------+
| | | | | | | | | | |I|V|V|A|V|R| |N| I |O|D|I|T|S|Z| |A| |P| |C|
| | | | | | | | | | |D|I|I|C|M|F| |T| P |F|F|F|F|F|F| |F| |F| |F|
| | | | | | | | | | |P|F| | | | | L | | | | | | | | | | |
+---------------------------------------------------------------+
```

The flags are described in Section 3.4.3 of Volume 1 of the SDM. To determine how each instruction affects the flags, see Appendix A of Volume 1 of the SDM.

The System Developer's Manual

The System Developer's Manual contains vast amounts of important information and is required reading for all assembly language programmers and backend compiler writers. The manual is split into several volumes; links to all volumes are here. Highlights from Volumes 1 and 2:

- **Volume 1:**
  - Chapter 1 - About this manual
  - Chapter 2 - History of the IA-32 and Intel 64 architectures, a description of many of the microarchitectures and processors and technologies
  - Chapter 3 - Basic execution environment
  - Chapter 4 - Data types
  - Chapter 5 - Instruction set summary. Lists all instructions and a brief (but not precise) description of each. Instructions are grouped into convenient categories.
  - Chapter 6 - Details on calls and returns, and exceptions
  - Chapter 7 - All about general purpose instructions
  - Chapter 8 - All about FPU instructions
  - Chapter 9 - All about MMX instructions
  - Chapter 10 - All about SSE instructions
  - Chapter 11 - All about SSE2 instructions
  - Chapter 12 - All about SSE3, SSSE3, SSE4 and AESNI instructions
  - Chapter 13 - XSAVE
  - Chapter 14 - All about AVX, FMA, and AVX2 instructions
  - Chapter 15 - AVX-512
  - Chapter 16 - All about transactional synchronization instructions
  - Chapter 17 - Memory protection extensions
  - Chapter 18 - All about I/O instructions
  - Chapter 19 - How to determine what processor you have, and what its features are
  - Appendix A - Shows which instructions affect which flags
  - Appendix B - Condition codes
  - Appendix C - Floating-point exceptions
  - Appendix D - Guidelines for writing x87 exception handlers
  - Appendix E - Guidelines for writing SIMD exception handlers
- **Volume 2:**
  - Chapter 1 - About this manual
  - Chapter 2 - Instruction formats

- Chapter 3-5 - Instruction set reference: full description, and encodings, of *every* instruction with names beginning with A — L
- Chapter 6 - Safer Mode Extensions Reference
- Appendix A - Opcode map
- Appendix B - Encoding summary
- Appendix C - Compiler intrinsics

NASM Tutorial



Yep, it's a tutorial.

CONTENTS

Scope of the Tutorial • Your First Program • Structure of a NASM Program • Details • Your First Few Instructions • The Three Kinds of Operands • Instructions with two memory operands are extremely rare • Defining Data and Reserving Space • Another Example • Using a C Library • Understanding Calling Conventions • Mixing C and Assembly Language • Conditional Instructions • Command Line Arguments • A Longer Example • Floating Point Instructions • Data Sections • Recursion • SIMD Parallelism • Saturated Arithmetic • Graphics • Local Variables and Stack Frames • Using NASM on macOS • Using NASM on Windows • Summary

Scope of the Tutorial

This tutorial will show you how to write assembly language programs on the x86-64 architecture.

You will write both (1) standalone programs and (2) programs that integrate with C.

Don't worry, we won't get too fancy.

Your First Program

Before learning any details, let's make sure you can type in and run programs.

Make sure both nasm and gcc are installed. Save one of the following programs as **hello.asm**, depending on your machine platform. Then run the program according to the given instructions.

If you are on a Linux-based OS:

hello.asm

```
; ------------------------------------------------------------------------------
; Writes "Hello, World" to the console using only system calls. Runs on 64-bit Linux only.
; To assemble and run:
;
;     nasm -felf64 hello.asm && ld hello.o && ./a.out
; ------------------------------------------------------------------------------

          global    _start

          section   .text
_start:   mov       rax, 1                  ; system call for write
          mov       rdi, 1                  ; file handle 1 is stdout
          mov       rsi, message            ; address of string to output
          mov       rdx, 13                 ; number of bytes
          syscall                           ; invoke operating system to do the write
          mov       rax, 60                 ; system call for exit
          xor       rdi, rdi                ; exit code 0
          syscall                           ; invoke operating system to exit

          section   .data
message:  db        "Hello, World", 10      ; note the newline at the end
```

```
$ nasm -felf64 hello.asm && ld hello.o && ./a.out
Hello, World
```

If you are on macOS:

hello.asm

```
; ------------------------------------------------------------------------------
; Writes "Hello, World" to the console using only system calls. Runs on 64-bit macOS only.
; To assemble and run:
;
;     nasm -fmacho64 hello.asm && ld hello.o && ./a.out
; ------------------------------------------------------------------------------

          global    start

          section   .text
start:    mov       rax, 0x02000004         ; system call for write
          mov       rdi, 1                  ; file handle 1 is stdout
```

```
        mov     rsi, message            ; address of string to output
        mov     rdx, 13                 ; number of bytes
        syscall                         ; invoke operating system to do the write
        mov     rax, 0x02000001         ; system call for exit
        xor     rdi, rdi                ; exit code 0
        syscall                         ; invoke operating system to exit

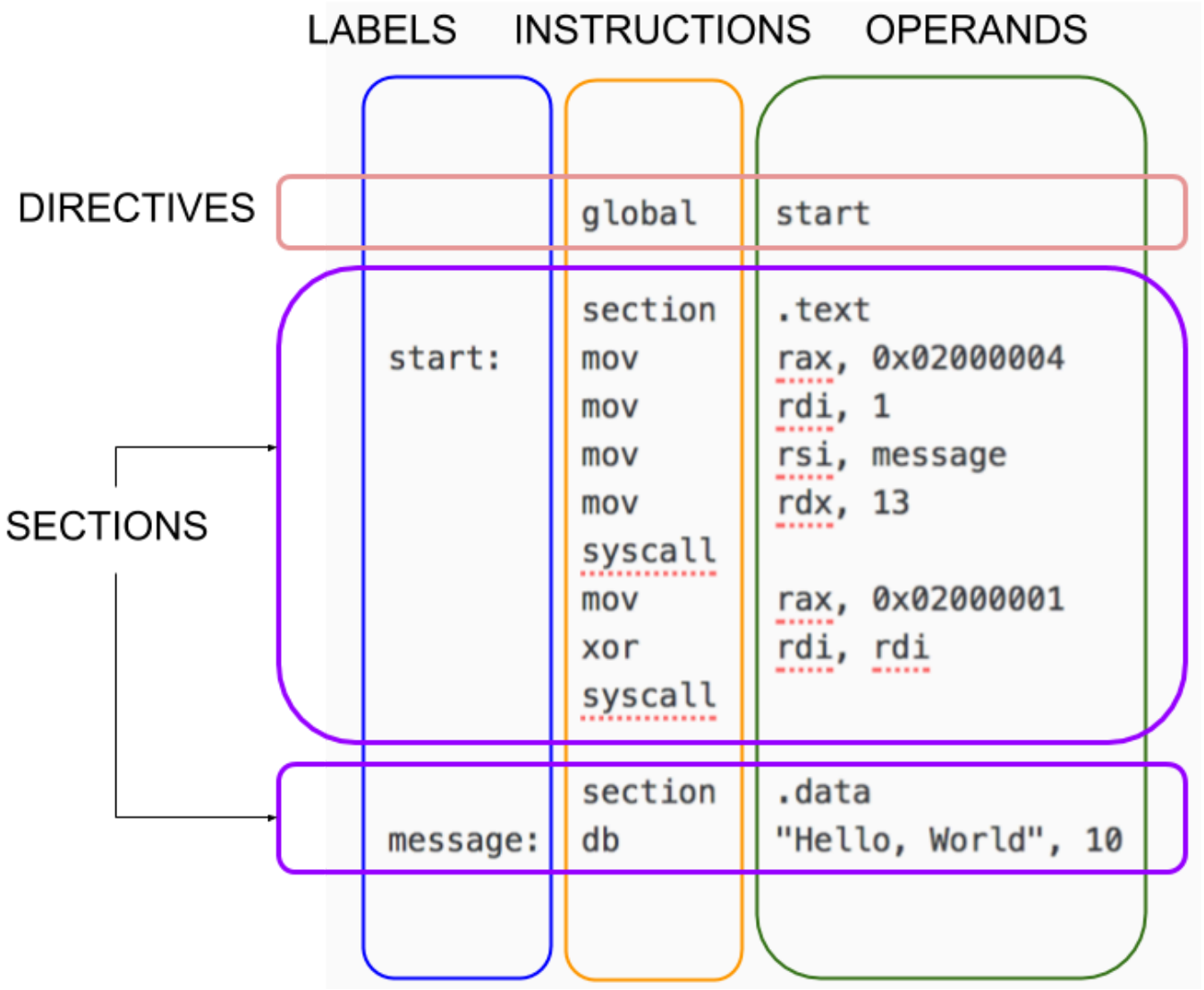
        section .data
message: db     "Hello, World", 10      ; note the newline at the end
```

```
$ nasm -fmacho64 hello.asm && ld hello.o && ./a.out
Hello, World
```

***Exercise**: Identify the differences between the two programs.*

Structure of a NASM Program

NASM is line-based. Most programs consist of **directives** followed by one or more **sections**. Lines can have an optional **label**. Most lines have an **instruction** followed by zero or more **operands**.



Generally, you put code in a section called `.text` and your constant data in a section called `.data`.

Details

NASM is an awesome assembler, but assembly language is complex. You need more than a tutorial. You need details. Lots of details. Be ready to consult:

- The NASM Manual, which is pretty good!
- The Intel Processor Manuals

Your First Few Instructions

There are hundreds of instructions. You can't learn them all at once. Just start with these:

Instruction	Description
mov x, y	$x \leftarrow y$
and x, y	$x \leftarrow x \wedge y$
or x, y	$x \leftarrow x \vee y$
xor x, y	$x \leftarrow x \oplus y$
add x, y	$x \leftarrow x + y$
sub x, y	$x \leftarrow x - y$
inc x	$x \leftarrow x + 1$
dec x	$x \leftarrow x - 1$
syscall n	Invoke operating system routine n
db	A pseudo-instruction that declares bytes that will be in memory when the program runs

The Three Kinds of Operands

It helps to know these cold. Get ready to memorize them. How can you remember things? Nicky Case can help you with this!

Register Operands

In this tutorial we only care about the integer registers, the flag register, and the xmm registers. (If you are familiar with the x86 architecture, you will know that this means we are skipping the FP, MMX, YMM, segment, control, debug, test, and protected mode registers.) Hopefully you have already been introduced to the x86-84 architecture, in which case, this is a quick review. The 16 integer registers are 64 bits wide and are called:

R0 aka RAX	R1 aka RCX	R2 aka RDX	R3 aka RBX	R4 aka RSP	R5 aka RBP	R6 aka RSI	R7 aka RDI	R8	R9	R10	R11	R12	R13	R14	R15

(Note that 8 of the registers have alternate names.) You can treat the lowest 32-bits of each register as a register itself but using these names:

R0D aka EAX	R1D aka ECX	R2D aka EDX	R3D aka EBX	R4D aka ESP	R5D aka EBP	R6D aka ESI	R7D aka EDI	R8D	R9D	R10D	R11D	R12D	R13D	R14D	R15D

You can treat the lowest 16-bits of each register as a register itself but using these names:

R0W aka AX	R1W aka CX	R2W aka DX	R3W aka BX	R4W aka SP	R5W aka BP	R6W aka SI	R7W aka DI	R8W	R9W	R10W	R11W	R12W	R13W	R14W	R15W

You can treat the lowest 8-bits of each register as a register itself but using these names:

R0B aka AL	R1B aka CL	R2B aka DL	R3B aka BL	R4B aka SPL	R5B aka BPL	R6B aka SIL	R7B aka DIL	R8B	R9B	R10B	R11B	R12B	R13B	R14B	R15B

For historical reasons, bits 15 through 8 of `R0` .. `R3` are named:

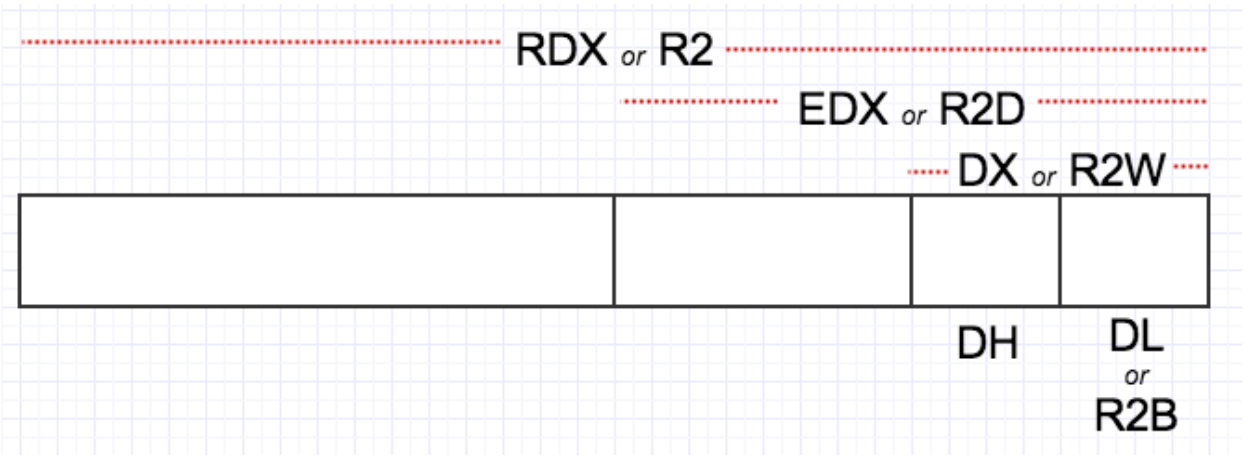
AH	CH	DH	BH

And finally, there are 16 XMM registers, each 128 bits wide, named:

XMM0	XMM1	XMM2	XMM3	XMM4	XMM5	XMM6	XMM7	XMM8	XMM9	XMM10	XMM11	XMM12

XMM13	XMM14	XMM15

Study this picture; hopefully it helps:



Memory Operands

These are the basic forms of addressing:

- `[ number ]`
- `[ reg ]`
- `[ reg + reg*scale ]` scale is 1, 2, 4, or 8 only
- `[ reg + number ]`

- `[ reg + reg*scale + number ]`

The number is called the **displacement**; the plain register is called the **base**; the register with the scale is called the **index**.

Examples:

```
[750]                ; displacement only
[rbp]                ; base register only
[rcx + rsi*4]        ; base + index * scale
[rbp + rdx]          ; scale is 1
[rbx - 8]            ; displacement is -8
[rax + rdi*8 + 500]  ; all four components
[rbx + counter]      ; uses the address of the variable 'counter' as the displacement
```

Immediate Operands

These can be written in many ways. Here are some examples from the official docs.

```
200          ; decimal
0200         ; still decimal - the leading 0 does not make it octal
0200d        ; explicitly decimal - d suffix
0d200        ; also decimal - 0d prefex
0c8h         ; hex - h suffix, but leading 0 is required because c8h looks like a var
0xc8         ; hex - the classic 0x prefix
0hc8         ; hex - for some reason NASM likes 0h
310q         ; octal - q suffix
0q310        ; octal - 0q prefix
11001000b    ; binary - b suffix
0b1100_1000  ; binary - 0b prefix, and by the way, underscores are allowed
```

Instructions with two memory operands are extremely rare

In fact, we'll not see any such instruction in this tutorial. Most of the basic instructions have only the following forms:

- **add** *reg*, *reg*
- **add** *reg*, *mem*
- **add** *reg*, *imm*

- **add** *mem*, *reg*
- **add** *mem*, *imm*

Defining Data and Reserving Space

These examples come from Chapter 3 of the docs. To place data in memory:

```
db    0x55                ; just the byte 0x55
db    0x55,0x56,0x57      ; three bytes in succession
db    'a',0x55            ; character constants are OK
db    'hello',13,10,'$'   ; so are string constants
dw    0x1234              ; 0x34 0x12
dw    'a'                 ; 0x61 0x00 (it's just a number)
dw    'ab'                ; 0x61 0x62 (character constant)
dw    'abc'               ; 0x61 0x62 0x63 0x00 (string)
dd    0x12345678          ; 0x78 0x56 0x34 0x12
dd    1.234567e20         ; floating-point constant
dq    0x123456789abcdef0  ; eight byte constant
dq    1.234567e20         ; double-precision float
dt    1.234567e20         ; extended-precision float
```

There are other forms; check the NASM docs. Later.

To reserve space (without initializing), you can use the following pseudo instructions. They should go in a section called `.bss` (you'll get an error if you try to use them in a `.text` section):

```
buffer:         resb    64              ; reserve 64 bytes
wordvar:        resw    1               ; reserve a word
realarray:      resq    10              ; array of ten reals
```

Another Example

Here's a macOS program to study:

triangle.asm

```
; ------------------------------------------------------------------------------
; This is an macOS console program that writes a little triangle of asterisks to standard
; output. Runs on macOS only.
```

```
;
;     nasm -fmacho64 triangle.asm && gcc hola.o && ./a.out
; -----------------------------------------------------------------------------

          global    start
          section   .text
start:
          mov       rdx, output             ; rdx holds address of next byte to write
          mov       r8, 1                   ; initial line length
          mov       r9, 0                   ; number of stars written on line so far
line:
          mov       byte [rdx], '*'         ; write single star
          inc       rdx                     ; advance pointer to next cell to write
          inc       r9                      ; "count" number so far on line
          cmp       r9, r8                  ; did we reach the number of stars for this line?
          jne       line                    ; not yet, keep writing on this line
lineDone:
          mov       byte [rdx], 10          ; write a new line char
          inc       rdx                     ; and move pointer to where next char goes
          inc       r8                      ; next line will be one char longer
          mov       r9, 0                   ; reset count of stars written on this line
          cmp       r8, maxlines            ; wait, did we already finish the last line?
          jng       line                    ; if not, begin writing this line
done:
          mov       rax, 0x02000004         ; system call for write
          mov       rdi, 1                  ; file handle 1 is stdout
          mov       rsi, output             ; address of string to output
          mov       rdx, dataSize           ; number of bytes
          syscall                           ; invoke operating system to do the write
          mov       rax, 0x02000001         ; system call for exit
          xor       rdi, rdi                ; exit code 0
          syscall                           ; invoke operating system to exit

          section   .bss
maxlines  equ       8
dataSize  equ       44
output:   resb      dataSize
```

```
$ nasm -fmacho64 triangle.asm && ld triangle.o && ./a.out
*
```

```
**
***
****
*****
******
*******
********
```

New things in this example:

- `cmp` does a comparison
- `je` jumps to a label if the previous comparison was equal. We also have `jne` (jump if not equal), `jl` (jump if less), `jnl` (jump if not less), `jg` (jump if greater), `jng` (jump if not greater), `jle` (jump if less or equal), `jnle` (jump if not less or equal), `jge` (jump if greater or equal), `jnge` (jump if not greater or equal), and many more.
- `equ` is actually not a real instruction. It simply defines an abbreviation for the assembler itself to use. (This is a profound idea.)
- The `.bss` section is for *writable* data.

Using a C Library

Writing standalone programs with just system calls is cool, but rare. We would like to use the good stuff in the C library.

Remember how in C execution "starts" at the function `main`? That's because the C library actually has the `_start` label inside itself! The code at `_start` does some initialization, then it calls `main`, then it does some clean up, then it issues the system call for exit. So you just have to implement `main`. We can do that in assembly!

If you have Linux, try this:

hola.asm

```
; ------------------------------------------------------------------------------
; Writes "Hola, mundo" to the console using a C library. Runs on Linux.
;
;     nasm -felf64 hola.asm && gcc hola.o && ./a.out
; ------------------------------------------------------------------------------

          global    main
          extern    puts
```

```
          section   .text
main:                                   ; This is called by the C library startup code
          mov       rdi, message        ; First integer (or pointer) argument in rdi
          call      puts                ; puts(message)
          ret                           ; Return from main back into C library wrapper
message:
          db        "Hola, mundo", 0    ; Note strings must be terminated with 0 in C
```

```
$ nasm -felf64 hola.asm && gcc hola.o && ./a.out
Hola, mundo
```

Under macOS, it will look a little different:

hola.asm

```
; ------------------------------------------------------------------------------
; This is an macOS console program that writes "Hola, mundo" on one line and then exits.
; It uses puts from the C library.  To assemble and run:
;
;     nasm -fmacho64 hola.asm && gcc hola.o && ./a.out
; ------------------------------------------------------------------------------

          global    _main
          extern    _puts

          section   .text
_main:    push      rbx                 ; Call stack must be aligned
          lea       rdi, [rel message]  ; First argument is address of message
          call      _puts               ; puts(message)
          pop       rbx                 ; Fix up stack before returning
          ret

          section   .data
message:  db        "Hola, mundo", 0    ; C strings need a zero byte at the end
```

```
$ nasm -fmacho64 hola.asm && gcc hola.o && ./a.out
Hola, mundo
```

In macOS land, C functions (or any function that is exported from one module to another, really) must be prefixed with underscores. The call stack must be aligned on a 16-byte boundary (more on this later). And when accessing named variables, a `rel` prefix is required.

Understanding Calling Conventions

How did we know the argument to `puts` was supposed to go in `RDI`? Answer: there are a number of conventions that are followed regarding calls.

When writing code for 64-bit Linux that integrates with a C library, you must follow the calling conventions explained in the AMD64 ABI Reference. You can also get this information from Wikipedia. The most important points are:

- From left to right, pass as many parameters as will fit in registers. The order in which registers are allocated, are:
  - For integers and pointers, `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`.
  - For floating-point (float, double), `xmm0`, `xmm1`, `xmm2`, `xmm3`, `xmm4`, `xmm5`, `xmm6`, `xmm7`.
- Additional parameters are pushed on the stack, right to left, and are to be *removed by the caller* after the call.
- After the parameters are pushed, the call instruction is made, so when the called function gets control, the return address is at `[rsp]`, the first memory parameter is at `[rsp+8]`, etc.
- **The stack pointer `rsp` must be aligned to a 16-byte boundary before making a call**. Fine, but the process of making a call pushes the return address (8 bytes) on the stack, so when a function gets control, `rsp` is not aligned. You have to make that extra space yourself, by pushing something or subtracting 8 from `rsp`.
- The only registers that the called function is required to preserve (the calle-save registers) are: `rbp`, `rbx`, `r12`, `r13`, `r14`, `r15`. All others are free to be changed by the called function.
- The callee is also supposed to save the control bits of the XMCSR and the x87 control word, but x87 instructions are rare in 64-bit code so you probably don't have to worry about this.
- Integers are returned in `rax` or `rdx:rax`, and floating point values are returned in `xmm0` or `xmm1:xmm0`.

Got that? No? What's need is more examples, and practice.

Here is a program that illustrates how registers have to be saved and restored:

fib.asm

```
; ------------------------------------------------------------------------
; A 64-bit Linux application that writes the first 90 Fibonacci numbers. To
; assemble and run:
```

```
;
;     nasm -felf64 fib.asm && gcc fib.o && ./a.out
; ------------------------------------------------------------------------

        global  main
        extern  printf

        section .text
main:
        push    rbx                     ; we have to save this since we use it

        mov     ecx, 90                 ; ecx will countdown to 0
        xor     rax, rax                ; rax will hold the current number
        xor     rbx, rbx                ; rbx will hold the next number
        inc     rbx                     ; rbx is originally 1
print:
        ; We need to call printf, but we are using rax, rbx, and rcx.  printf
        ; may destroy rax and rcx so we will save these before the call and
        ; restore them afterwards.

        push    rax                     ; caller-save register
        push    rcx                     ; caller-save register

        mov     rdi, format             ; set 1st parameter (format)
        mov     rsi, rax                ; set 2nd parameter (current_number)
        xor     rax, rax                ; because printf is varargs

        ; Stack is already aligned because we pushed three 8 byte registers
        call    printf                  ; printf(format, current_number)

        pop     rcx                     ; restore caller-save register
        pop     rax                     ; restore caller-save register

        mov     rdx, rax                ; save the current number
        mov     rax, rbx                ; next number is now current
        add     rbx, rdx                ; get the new next number
        dec     ecx                     ; count down
        jnz     print                   ; if not done counting, do some more
```

```
        pop     rbx                     ; restore rbx before returning
        ret
format:
        db  "%20ld", 10, 0
```

```
$ nasm -felf64 fib.asm && gcc fib.o && ./a.out
                   0
                   1
                   1
                   2
                   .
                   .
                   .
  679891637638612258
 1100087778366101931
 1779979416004714189
```

We just saw some new instructions:

Instruction	Description
push *x*	Decrement `rsp` by the size of the operand, then store *x* in `[rsp]`
pop *x*	Move `[rsp]` into *x*, then increment `rsp` by the size of the operand
jnz *label*	If the processor's Z (zero) flag is set, jump to the given label
call *label*	Push the address of the next instruction, then jump to the label
ret	Pop into the instruction pointer

Mixing C and Assembly Language

This program is just a simple function that takes in three integer parameters and returns the maximum value.

maxofthree.asm

```
; ------------------------------------------------------------------------
; A 64-bit function that returns the maximum value of its three 64-bit integer
; arguments.  The function has signature:
;
;   int64_t maxofthree(int64_t x, int64_t y, int64_t z)
;
; Note that the parameters have already been passed in rdi, rsi, and rdx.  We
; just have to return the value in rax.
```

```
; ------------------------------------------------------------------------

        global  maxofthree
        section .text
maxofthree:
        mov     rax, rdi                ; result (rax) initially holds x
        cmp     rax, rsi                ; is x less than y?
        cmovl   rax, rsi                ; if so, set result to y
        cmp     rax, rdx                ; is max(x,y) less than z?
        cmovl   rax, rdx                ; if so, set result to z
        ret                             ; the max will be in rax
```

Here is a C program that calls the assembly language function.

callmaxofthree.c

```c
/*
 * A small program that illustrates how to call the maxofthree function we wrote in
 * assembly language.
 */

#include <stdio.h>
#include <inttypes.h>

int64_t maxofthree(int64_t, int64_t, int64_t);

int main() {
    printf("%ld\n", maxofthree(1, -4, -7));
    printf("%ld\n", maxofthree(2, -6, 1));
    printf("%ld\n", maxofthree(2, 3, 1));
    printf("%ld\n", maxofthree(-2, 4, 3));
    printf("%ld\n", maxofthree(2, -6, 5));
    printf("%ld\n", maxofthree(2, 4, 6));
    return 0;
}
```

```
$ nasm -felf64 maxofthree.asm && gcc callmaxofthree.c maxofthree.o && ./a.out
1
2
3
4
5
6
```

Conditional Instructions

After an arithmetic or logic instruction, or the compare instruction, `cmp`, the processor sets or clears bits in its `rflags`. The most interesting flags are:

- `s` (sign)
- `z` (zero)
- `c` (carry)
- `o` (overflow)

So after doing, say, an addition instruction, we can perform a jump, move, or set, based on the new flag settings. For example:

Instruction	Description
`jz` L	Jump to label L if the result of the operation was zero
`cmovno` x, y	$x \leftarrow y$ if the last operation did not overflow
`setc` x	$x \leftarrow 1$ if the last operation had a carry, but $x \leftarrow 0$ otherwise (x must be a byte-size register or memory location)

The conditional instructions have three base forms: `j` for conditional jump, `cmov` for conditional move, and `set` for conditional set. The suffix of the instruction has one of the 30 forms: `s ns z nz c nc o no p np pe po e ne l nl le nle g ng ge nge a na ae nae b nb be nbe`.

Command Line Arguments

You know that in C, `main` is just a plain old function, and it has a couple parameters of its own:

```
int main(int argc, char** argv)
```

So, you guessed it, `argc` will end up in `rdi`, and `argv` (a pointer) will end up in `rsi`. Here is a program that uses this fact to simply echo the command line arguments to a program, one per line:

echo.asm

```
; ------------------------------------------------------------------------
; A 64-bit program that displays its command line arguments, one per line.
;
```

```
; On entry, rdi will contain argc and rsi will contain argv.
; -----------------------------------------------------------------------

        global  main
        extern  puts
        section .text
main:
        push    rdi                     ; save registers that puts uses
        push    rsi
        sub     rsp, 8                  ; must align stack before call

        mov     rdi, [rsi]              ; the argument string to display
        call    puts                    ; print it

        add     rsp, 8                  ; restore %rsp to pre-aligned value
        pop     rsi                     ; restore registers puts used
        pop     rdi

        add     rsi, 8                  ; point to next argument
        dec     rdi                     ; count down
        jnz     main                    ; if not done counting keep going

        ret
```

```
$ nasm -felf64 echo.asm && gcc echo.o && ./a.out dog 22 -zzz "hi there"
./a.out
dog
22
-zzz
hi there
```

A Longer Example

Note that as far as the C Library is concerned, command line arguments are always strings. If you want to treat them as integers, call `atoi`. Here's a neat program to compute x^y.

power.asm

```
; ------------------------------------------------------------------------
; A 64-bit command line application to compute x^y.
;
; Syntax: power x y
; x and y are (32-bit) integers
; ------------------------------------------------------------------------

        global  main
        extern  printf
        extern  puts
        extern  atoi

        section .text
main:
        push    r12                     ; save callee-save registers
        push    r13
        push    r14
        ; By pushing 3 registers our stack is already aligned for calls

        cmp     rdi, 3                  ; must have exactly two arguments
        jne     error1

        mov     r12, rsi                ; argv

; We will use ecx to count down form the exponent to zero, esi to hold the
; value of the base, and eax to hold the running product.

        mov     rdi, [r12+16]           ; argv[2]
        call    atoi                    ; y in eax
        cmp     eax, 0                  ; disallow negative exponents
        jl      error2
        mov     r13d, eax               ; y in r13d

        mov     rdi, [r12+8]            ; argv
        call    atoi                    ; x in eax
        mov     r14d, eax               ; x in r14d

        mov     eax, 1                  ; start with answer = 1
check:
```

```
        test    r13d, r13d              ; we're counting y downto 0
        jz      gotit                   ; done
        imul    eax, r14d               ; multiply in another x
        dec     r13d
        jmp     check
gotit:                                  ; print report on success
        mov     rdi, answer
        movsxd  rsi, eax
        xor     rax, rax
        call    printf
        jmp     done
error1:                                 ; print error message
        mov     edi, badArgumentCount
        call    puts
        jmp     done
error2:                                 ; print error message
        mov     edi, negativeExponent
        call    puts
done:                                   ; restore saved registers
        pop     r14
        pop     r13
        pop     r12
        ret


answer:
        db      "%d", 10, 0
badArgumentCount:
        db      "Requires exactly two arguments", 10, 0
negativeExponent:
        db      "The exponent may not be negative", 10, 0
```

```
$ nasm -felf64 power.asm && gcc -o power power.o
$ ./power 2 19
524288
$ ./power 3 -8
The exponent may not be negative
$ ./power 1 500
1
$ ./power 1
Requires exactly two arguments
```

Floating Point Instructions

Floating-point arguments go int the xmm registers. Here is a simple function for summing the values in a double array:

sum.asm

```
; ------------------------------------------------------------------------
; A 64-bit function that returns the sum of the elements in a floating-point
; array. The function has prototype:
;
;   double sum(double[] array, uint64_t length)
; ------------------------------------------------------------------------

        global  sum
        section .text
sum:
        xorpd   xmm0, xmm0              ; initialize the sum to 0
        cmp     rsi, 0                  ; special case for length = 0
        je      done
next:
        addsd   xmm0, [rdi]             ; add in the current array element
        add     rdi, 8                  ; move to next array element
        dec     rsi                     ; count down
        jnz     next                    ; if not done counting, continue
done:
        ret                             ; return value already in xmm0
```

Note the floating point instructions have an `sd` suffix; that's the most common one, but we'll see some other ones later. Here is a C program that calls it:

callsum.c

```c
/*
 * Illustrates how to call the sum function we wrote in assembly language.
 */

#include <stdio.h>
#include <inttypes.h>

double sum(double[], uint64_t);
```

```c
int main() {
    double test[] = {
        40.5, 26.7, 21.9, 1.5, -40.5, -23.4
    };
    printf("%20.7f\n", sum(test, 6));
    printf("%20.7f\n", sum(test, 2));
    printf("%20.7f\n", sum(test, 0));
    printf("%20.7f\n", sum(test, 3));
    return 0;
}
```

```
$ nasm -felf64 sum.asm && gcc sum.o callsum.c && ./a.out
          26.7000000
          67.2000000
           0.0000000
          89.1000000
```

Data Sections

The text section is read-only on most operating systems, so you might find the need for a data section. On most operating systems, the data section is only for initialized data, and you have a special .bss section for uninitialized data. Here is a program that averages the command line arguments, expected to be integers, and displays the result as a floating point number.

average.asm

```
; ------------------------------------------------------------------------
; 64-bit program that treats all its command line arguments as integers and
; displays their average as a floating point number.  This program uses a data
; section to store intermediate results, not that it has to, but only to
; illustrate how data sections are used.
; ------------------------------------------------------------------------

        global  main
        extern  atoi
        extern  printf
        default rel

        section .text
```

```
main:
        dec     rdi                     ; argc-1, since we don't count program name
        jz      nothingToAverage
        mov     [count], rdi            ; save number of real arguments
accumulate:
        push    rdi                     ; save register across call to atoi
        push    rsi
        mov     rdi, [rsi+rdi*8]        ; argv[rdi]
        call    atoi                    ; now rax has the int value of arg
        pop     rsi                     ; restore registers after atoi call
        pop     rdi
        add     [sum], rax              ; accumulate sum as we go
        dec     rdi                     ; count down
        jnz     accumulate              ; more arguments?
average:
        cvtsi2sd xmm0, [sum]
        cvtsi2sd xmm1, [count]
        divsd   xmm0, xmm1              ; xmm0 is sum/count
        mov     rdi, format             ; 1st arg to printf
        mov     rax, 1                  ; printf is varargs, there is 1 non-int argument

        sub     rsp, 8                  ; align stack pointer
        call    printf                  ; printf(format, sum/count)
        add     rsp, 8                  ; restore stack pointer

        ret

nothingToAverage:
        mov     rdi, error
        xor     rax, rax
        call    printf
        ret

        section .data
count:  dq      0
sum:    dq      0
format: db      "%g", 10, 0
error:  db      "There are no command line arguments to average", 10, 0
```

```
$ nasm -felf64 average.asm && gcc average.o && ./a.out 19 8 21 -33
3.75
$ nasm -felf64 average.asm && gcc average.o && ./a.out
There are no command line arguments to average
```

This program highlighted some processor instructions that convert between integers and floating point values. A few of the most common are:

Instruction	Description
cvtsi2sd *xmmreg*, *r/m32*	*xmmreg*[63..0] ← intToDouble(*r/m32*)
cvtsi2ss *xmmreg*, *r/m32*	*xmmreg*[31..0] ← intToFloat(*r/m32*)
cvtsd2si *reg32*, *xmmr/m64*	*reg32* ← doubleToInt(*xmmr/m64*)
cvtss2si *reg32*, *xmmr/m32*	*reg32* ← floatToInt(*xmmr/m32*)

Recursion

Perhaps surprisingly, there's nothing out of the ordinary required to implement recursive functions. You just have to be careful to save registers, as usual. Pushing and popping around the recursive call is a typical strategy.

factorial.asm

```
; ------------------------------------------------------------------------
; An implementation of the recursive function:
;
;   uint64_t factorial(uint64_t n) {
;       return (n <= 1) ? 1 : n * factorial(n-1);
;   }
; ------------------------------------------------------------------------

        global  factorial

        section .text
factorial:
        cmp     rdi, 1                  ; n <= 1?
        jnbe    L1                      ; if not, go do a recursive call
        mov     rax, 1                  ; otherwise return 1
        ret
```

```
L1:
        push    rdi                     ; save n on stack (also aligns %rsp!)
        dec     rdi                     ; n-1
        call    factorial               ; factorial(n-1), result goes in %rax
        pop     rdi                     ; restore n
        imul    rax, rdi                ; n * factorial(n-1), stored in %rax
        ret
```

An example caller:

callfactorial.c

```c
/*
 * An application that illustrates calling the factorial function defined elsewhere.
 */

#include <stdio.h>
#include <inttypes.h>

uint64_t factorial(uint64_t n);

int main() {
    for (uint64_t i = 0; i < 20; i++) {
        printf("factorial(%2lu) = %lu\n", i, factorial(i));
    }
    return 0;
}
```

```
$ nasm -felf64 factorial.asm && gcc -std=c99 factorial.o callfactorial.c && ./a.out
factorial( 0) = 1
factorial( 1) = 1
factorial( 2) = 2
factorial( 3) = 6
factorial( 4) = 24
factorial( 5) = 120
factorial( 6) = 720
factorial( 7) = 5040
factorial( 8) = 40320
factorial( 9) = 362880
factorial(10) = 3628800
factorial(11) = 39916800
factorial(12) = 479001600
factorial(13) = 6227020800
factorial(14) = 87178291200
factorial(15) = 1307674368000
factorial(16) = 20922789888000
factorial(17) = 355687428096000
factorial(18) = 6402373705728000
factorial(19) = 121645100408832000
```

SIMD Parallelism

The XMM registers can do arithmetic on floating point values one operation at a time (scalar) or multiple operations at a time (packed). The operations have the form:

op xmmreg_or_memory, xmmreg

For floating point addition, the instructions are:

Instruction	Description
addpd	Do two double-precision additions in parallel (add **p**acked **d**ouble)
addsd	Do just one double-precision addition, using the low 64-bits of the register (add **s**calar **d**ouble)
addps	Do four single-precision additions in parallel (add **p**acked **s**ingle)
addss	Do just one single-precision addition, using the low 32-bits of the register (add **s**calar **s**ingle)

Here's a function that adds four floats at once:

add_four_floats.asm

```
; void add_four_floats(float x[4], float y[4])
; x[i] += y[i] for i in range(0..4)

        global   add_four_floats
        section  .text

add_four_floats:
        movdqa   xmm0, [rdi]            ; all four values of x
        movdqa   xmm1, [rsi]            ; all four values of y
        addps    xmm0, xmm1             ; do all four sums in one shot
        movdqa   [rdi], xmm0
        ret
```

and a caller:

test_add_four_floats.c

```
#include <stdio.h>
void add_four_floats(float[], float[]);
```

```
int main() {
    float x[] = {-29.750, 244.333, 887.29, 48.1E22};
    float y[] = {29.750,  199.333, -8.29,  22.1E23};
    add_four_floats(x, y);
    printf("%f\n%f\n%f\n%f\n", x[0], x[1], x[2], x[3]);
    return 0;
}
```

Also see this nice little x86 floating-point slide deck from Ray Seyfarth.

Saturated Arithmetic

The XMM registers can also do arithmetic on integers. The instructions have the form:

op xmmreg_or_memory, xmmreg

For integer addition, the instructions are:

Instruction	Description
paddb	Do 16 byte-additions
paddw	Do 8 word-additions
paddd	Do 4 dword-additions
paddq	Do 2 qword-additions
paddsb	Do 16 byte-additions with signed saturation (80..7F)
paddsw	Do 8 word-additions with signed saturation (8000..7F)
paddusb	Do 16 byte-additions with unsigned saturation (00..FF)
paddusw	Do 8 word-additions with unsigned saturation (00..FFFF)

Here's an example. It also illustrates how you load the XMM registers. You can't load immediate values; you have to use `movaps` to move from memory. There are other ways, but we're not covering everything in this tutorial.

satexample.asm

```
; ------------------------------------------------------------------------------
; Example of signed saturated arithmetic.
; ------------------------------------------------------------------------------
```

```
        global  main
        extern  printf

        section .text
main:
        push    rbp
        movaps  xmm0, [arg1]
        movaps  xmm1, [arg2]
        paddsw  xmm0, xmm1
        movaps  [result], xmm0

        lea     rdi, [format]
        mov     esi, dword [result]
        mov     edx, dword [result+4]
        mov     ecx, dword [result+8]
        mov     r8d, dword [result+12]
        xor     rax, rax
        call    printf
        pop     rbp
        ret
        section .data
        align   16
arg1:   dw      0x3544,0x24FF,0x7654,0x9A77,0xF677,0x9000,0xFFFF,0x0000
arg2:   dw      0x7000,0x1000,0xC000,0x1000,0xB000,0xA000,0x1000,0x0000
result: dd      0, 0, 0, 0
format: db      '%x%x%x%x',10,0
```

Graphics

Any C program can be “ported” to assembly language. That goes for graphics programs, too.

> ***This program probably does not work.***
>
> *I last tested this in 2003. Back in the old-school OpenGL days. Used Win32. Pre-GLSL days. Used GLUT. I haven’t had access to a Windows box in a while and I’m not even sure it will work anymore.*

This is presented here for historical interest only. If you can modify it to work under modern OpenGL, please let me know. I'll update the program and cite your contribution, of course!

triangle.asm

```
; ------------------------------------------------------------------------
; triangle.asm
;
; A very simple *Windows* OpenGL application using the GLUT library.  It
; draws a nicely colored triangle in a top-level application window.  One
; interesting thing is that the Windows GL and GLUT functions do NOT use the
; C calling convention; instead they use the "stdcall" convention which is
; like C except that the callee pops the parameters.
; ------------------------------------------------------------------------

        global  _main
        extern  _glClear@4
        extern  _glBegin@4
        extern  _glEnd@0
        extern  _glColor3f@12
        extern  _glVertex3f@12
        extern  _glFlush@0
        extern  _glutInit@8
        extern  _glutInitDisplayMode@4
        extern  _glutInitWindowPosition@8
        extern  _glutInitWindowSize@8
        extern  _glutCreateWindow@4
        extern  _glutDisplayFunc@4
        extern  _glutMainLoop@0

        section .text
title:  db      'A Simple Triangle', 0
zero:   dd      0.0
one:    dd      1.0
half:   dd      0.5
neghalf:dd      -0.5

display:
        push    dword 16384
        call    _glClear@4              ; glClear(GL_COLOR_BUFFER_BIT)
```

```
        push    dword 9
        call    _glBegin@4              ; glBegin(GL_POLYGON)
        push    dword 0
        push    dword 0
        push    dword [one]
        call    _glColor3f@12           ; glColor3f(1, 0, 0)
        push    dword 0
        push    dword [neghalf]
        push    dword [neghalf]
        call    _glVertex3f@12          ; glVertex(-.5, -.5, 0)
        push    dword 0
        push    dword [one]
        push    dword 0
        call    _glColor3f@12           ; glColor3f(0, 1, 0)
        push    dword 0
        push    dword [neghalf]
        push    dword [half]
        call    _glVertex3f@12          ; glVertex(.5, -.5, 0)
        push    dword [one]
        push    dword 0
        push    dword 0
        call    _glColor3f@12           ; glColor3f(0, 0, 1)
        push    dword 0
        push    dword [half]
        push    dword 0
        call    _glVertex3f@12          ; glVertex(0, .5, 0)
        call    _glEnd@0                ; glEnd()
        call    _glFlush@0              ; glFlush()
        ret

_main:
        push    dword [esp+8]           ; push argv
        lea     eax, [esp+8]            ; get addr of argc (offset changed :-)
        push    eax
        call    _glutInit@8             ; glutInit(&argc, argv)
        push    dword 0
        call    _glutInitDisplayMode@4
        push    dword 80
        push    dword 80
```

```
        call    _glutInitWindowPosition@8
        push    dword 300
        push    dword 400
        call    _glutInitWindowSize@8
        push    title
        call    _glutCreateWindow@4
        push    display
        call    _glutDisplayFunc@4
        call    _glutMainLoop@0
        ret
```

Local Variables and Stack Frames

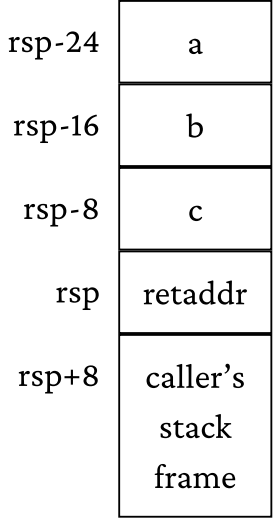
First, please read Eli Bendersky's article. That overview is more complete than my brief notes.

When a function is called the caller will first put the parameters in the correct registers then issue the `call` instruction. Additional parameters beyond those covered by the registers will be pushed on the stack prior to the call. The call instruction puts the return address on the top of stack. So if you have the function:

```
int64_t example(int64_t x, int64_t y) {
    int64_t a, b, c;
    b = 7;
    return x * b + y;
}
```

Then on entry to the function, x will be in edi, y will be in esi, and the return address will be on the top of the stack. Where can we put the local variables? An easy choice is on the stack itself, though if you have enough regsters, use them! Registers tend to be faster anyway.

If you are running on a machine that respect the standard ABI, you can leave rsp where it is and access the "extra parameters" and the local variables directly from rsp for example:



So our function looks like this:

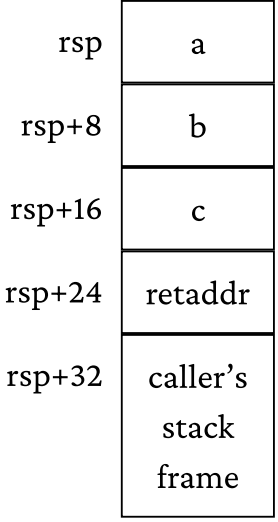
```
        global  example
        section .text
example:
        mov     qword [rsp-16], 7
        mov     rax, rdi
        imul    rax, [rsp+8]
        add     rax, rsi
        ret
```

If our function were to make another call, you would have to adjust rsp to get out of the way at that time.

On Windows you can't use this scheme because if an interrupt were to occur, everything above the stack pointer gets plastered. This doesn't happen on most other operating systems because there is a **red zone** of 128 bytes past the stack pointer which is safe from these things. In this case, you can make room on the stack immediately:

```
example:
        sub rsp, 24
```

so our stack looks like this:



Here's the function now. Note that we have to remember to replace the stack pointer before returning!

```
        global  example
        section .text
example:
        sub     rsp, 24
        mov     qword [rsp+8], 7
        mov     rax, rdi
        imul    rax, [rsp+8]
        add     rax, rsi
        add     rsp, 24
        ret
```

Using NASM on macOS

Hopefully you've gone through the whole tutorial above using a Linux-based operating system (or perhaps more correctly, and ELF64 system). There are pretty much only five thing to know to get these examples working under a 64-bit macOS system:

- This object file format is `macho64`, not `elf64`.
- The system call numbers are *totally different*.
- Symbols shared between modules will be prefixed by underscores.
- It seems that the gcc linker in macOS doesn't allow absolute addressing unless you tweak some settings. So add `default rel` when you are referencing labeled memory locations, and always use `lea` to get your addresses.
- Also, it appears that sometimes under Linux, the 16-bit stack alignment requirement is not enforced, but it appears to be *always* enforced under macOS.

So here's the average program from above, written for macOS.

average.asm

```
; ------------------------------------------------------------------------
; 64-bit program that treats all its command line arguments as integers and
; displays their average as a floating point number.  This program uses a data
; section to store intermediate results, not that it has to, but only to
; illustrate how data sections are used.
;
; Designed for OS X.  To assemble and run:
;
;     nasm -fmacho64 average.asm && gcc average.o && ./a.out
; ------------------------------------------------------------------------

        global  _main
        extern  _atoi
        extern  _printf
        default rel

        section .text
_main:
        push    rbx                 ; we don't ever use this, but it is necesary
                                    ; to align the stack so we can call stuff
        dec     rdi                 ; argc-1, since we don't count program name
        jz      nothingToAverage
        mov     [count], rdi        ; save number of real arguments
accumulate:
        push    rdi                 ; save register across call to atoi
        push    rsi
        mov     rdi, [rsi+rdi*8]    ; argv[rdi]
        call    _atoi               ; now rax has the int value of arg
        pop     rsi                 ; restore registers after atoi call
        pop     rdi
        add     [sum], rax          ; accumulate sum as we go
        dec     rdi                 ; count down
        jnz     accumulate          ; more arguments?
average:
        cvtsi2sd xmm0, [sum]
        cvtsi2sd xmm1, [count]
        divsd   xmm0, xmm1          ; xmm0 is sum/count
        lea     rdi, [format]       ; 1st arg to printf
```

```
        mov     rax, 1                  ; printf is varargs, there is 1 non-int argument
        call    _printf                 ; printf(format, sum/count)
        jmp     done

nothingToAverage:
        lea     rdi, [error]
        xor     rax, rax
        call    _printf

done:
        pop     rbx                     ; undoes the stupid push at the beginning
        ret

        section .data
count:  dq      0
sum:    dq      0
format: db      "%g", 10, 0
error:  db      "There are no command line arguments to average", 10, 0
```

```
$ nasm -fmacho64 average.asm && gcc average.o && ./a.out
There are no command line arguments to average
$ nasm -fmacho64 average.asm && gcc average.o && ./a.out 54.3
54
$ nasm -fmacho64 average.asm && gcc average.o && ./a.out 54.3 -4 -3 -25 455.1111
95.4
```

Using NASM on Windows

I'm not sure what the system calls are on Windows, but I do know that if you want to assemble and link with the C library, you have to understand the x64 conventions. Read them. You will learn such things as:

- The first four integer parameters are passed in RCX, RDX, R8, and R9. The rest are to be pushed on the stack.
- The callee must preserve RBX, RBP, RDI, RSI, RSP, R12, R13, R14, and R15.
- The first four floating point parameters are passed in, you guessed it, XMM0, XMM1, XMM2, and XMM3.
- Return values go in RAX or XMM0.

IMPORTANT: There's one thing that's really hard to find in any documentation: the x64 calling convention requires you to allocate 32 bytes of shadow space before each call, and remove it after your call. This means

your “hello world” program looks like this:

hello.asm

```
; ------------------------------------------------------------------------------
; This is a Win64 console program that writes "Hello" on one line and then exits.  It
; uses puts from the C library.  To assemble and run:
;
;     nasm -fwin64 hello.asm && gcc hello.obj && a
; ------------------------------------------------------------------------------

        global  main
        extern  puts
        section .text
main:
        sub     rsp, 28h                ; Reserve the shadow space
        mov     rcx, message            ; First argument is address of message
        call    puts                    ; puts(message)
        add     rsp, 28h                ; Remove shadow space
        ret
message:
        db      'Hello', 0              ; C strings need a zero byte at the end
```

Did you notice we actually reserved 40 bytes? Thirty-two bytes of shadow space is a minimum requirement. In our `main` function, we are calling another function, so our stack must be aligned on a 16-byte boundary. When `main` is called, the return address (8 bytes) was pushed, so we have to “add” an extra 8 bytes to the shadow space.

Summary

We’ve covered:

- ✅ How to run a NASM program
- ✅ The structure of a NASM program
- ✅ The most basic instructions
- ✅ Instruction formats
- ✅ Mixing C and assembly language
- ✅ Floating point, saturated arithmetic, and parallel instructions

✅ Calls, calling conventions, and recursion
✅ Some platform-specific details

Runtime Systems

What do we need to know, and what do we need, when our programs are running?

CONTENTS

Runtime Memory Models • Static Storage • Stack Storage • Heap Storage • Garbaage Collection • Process Management • Thread Management • Summary

Runtime Memory Models

TODO

Static Storage

TODO

Stack Storage

Each invocation of a subroutine has an **activation record**, or **frame** associated with it at runtime, holding some or all of the following:

- Arguments (parameters)
- A link to the caller's stack frame (the *dynamic link*)

- In a language with nested subroutines, a mechanism for accessing non-locals, which could be one of:
  - A static link, or
  - A pointer to the most recent frame at this level, or
  - The whole display
- The return address
- Local variables
- Copies of callee-save registers, if needed
- Temporaries
- Spilled registers

Note that some of the stack frame may be in registers and some of it may be in memory.

Activation records are also called **stack frames**. When the runtime supports closures or concurrency, something like a **spaghetti stack** is required.

Calling Conventions

Calling conventions specify:

- Order of evaluation of arguments (left-to-right, right-to-left, in parallel, unspecified...)
- Where the arguments go (run-time stack, registers, ...)
- Which registers are allowed for arguments, which registers are to be saved by the caller and which by the callee
- Whether the caller or callee has to clean up the stack

Example: C on the Intel64

TODO

Historical Example: C on the IA-32

The C-calling convention on the IA-32 is:

- Arguments are evaluated right to left and pushed on the stack
- Callee must save registers ebx, ebp, esi, edi if it uses them
- Caller must save every other register that it wants preserved
- Return value goes in eax, edx:eax, or st[0]

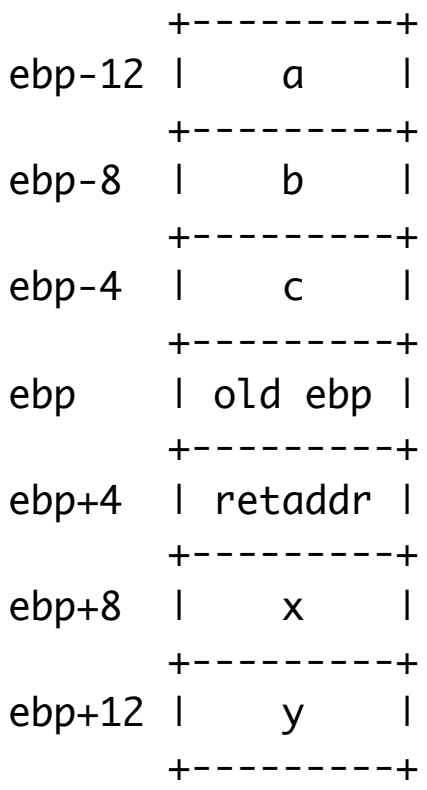
- Register ebp is used to point to the previous stack frame, so the old value of ebp is (almost) always saved.
- Space for local variables in the callee is allocated above the saved ebp.

```
int example(int x, int y) {
  int a, b, c;
  ...
  f(y, a, b, b, x);
  ...
}
```

The prolog/epilog for this function would look like:

```
    push    ebp          ; must save old ebp
    mov ebp, esp         ; point ebp to this frame
    sub esp, ___         ; make space for locals
    ...
    mov esp, ebp         ; clean up locals
    pop ebp           ; restore old ebp
    ret
```

with stack frame:



Example: SPARC

The SPARC uses **register windows**. You can read about these at Wikipedia. Let's start with an example:

```
! ----------------------------------------------------------------------
! writeint
!
! A small program that contains a function to write an unsigned integer value
! to standard output.
! ----------------------------------------------------------------------

        .global _start

        .text

! writeint(unsigned int n) writes n to standard output as text. It is
! inefficient since it is recursive and invokes a system call to write
! each digit.
!
! It is equivalent to the C function
!
!   void writeint(unsigned n) {
!       if (n >= 10) writeint(n / 10);
!       putchar(n + '0');
!   }

writeint:
    save %sp, -96, %sp

    mov 10, %l0                 ! Keep 10 in %l0
    cmp %i0, %l0                ! Is it less than 10?
    bl  writedigit              ! If so just write the digit
    nop
    mov %g0, %y
    udiv %i0, %l0, %l1          ! n / 10 -> %l1
    umul %l1, %l0, %l2          ! n * (n/10) -> %l2
    sub %i0, %l2, %l2           ! n mod 10 -> %l2

    call writeint
    mov %l1, %o0                ! call writeint(n / 10)

    mov %l2, %i0                ! Put digit into %i0 before writing

writedigit:                     ! Writes the digit in %i0
```

```
    add 48, %i0, %l1            ! Character value of the digit
    set digit, %o1
    stb %l1, [%o1]              ! Must be in memory to write
    mov 4, %g1                  ! System call write
    mov 1, %o0                  ! stdout
    mov 1, %o2                  ! number of characters to write
    ta  0

    ret
    restore

! A starting point for the program that just makes a couple of calls
! to the writeint function.

_start:
    call    writeint        ! writeint(5)
    mov 5, %o0

    call    writeint        ! writeint(1312)
    mov 1312, %o0

    sethi   %hi(7308841), %o0   ! writeint(7308841)
    call    writeint
    or  %o0, %lo(7308841), %o0

    mov 1, %g1
    ta  0

    .data
    .align  4
digit:  .skip   4
```

TODO - describe this example

Leaf Subroutines

A leaf subroutine makes no calls. A good compiler can generate code to make a leaf subroutine's activation use the stack frame of its caller. The SPARC has direct support for this. Example:

```
! write_stars(n) writes n stars followed by a newline

write_stars:
    mov %o0, %l0        ! save the argument in %l0
```

```
    mov 4, %g1          ! syscall write
    mov 1, %o0          ! stdout
    set star, %o1       ! message to write
    mov 1, %o2          ! one character in message
write_star:
    tst %l0             ! no more characters to write
    be  write_newline   ! write terminating newline if so
    nop
    ta  0
    ba  write_star      ! go see if more to write
    dec %l0             ! count off one more char as written
write_newline:
    set newline, %o1
    retl
    ta  0               ! write the newline (before returning)
star:
    .ascii  "*"
newline:
    .ascii  "\n"
```

TODO - describe this example

Inlining

A compiler inlines a subroutine call when it replaces the call with the body of the called subroutine (with all argument-parameter bindings resolved, of course). If all calls are inlined, there is no need to generate separate code for the subroutine. Inlining should be faster in general, since there is no call/return overhead (which is substantial for short subroutines). But if there are thousands of such calls you can end up with more compiled code.

And compilers have to be careful when inlining recursive subroutines.

Non Local Lexical Variables

Given:

```
function f(a, b) {
    var x = 1;

    function g(a, y) {
        var z = 3;
```

```
        function h(z) {
            print x;
        }

        function k(c) {
            var x = c;
            if (x < 5) k(c+1); else h(1);
        }

        if (a == 0) g(1, 0); else k(2);
    }

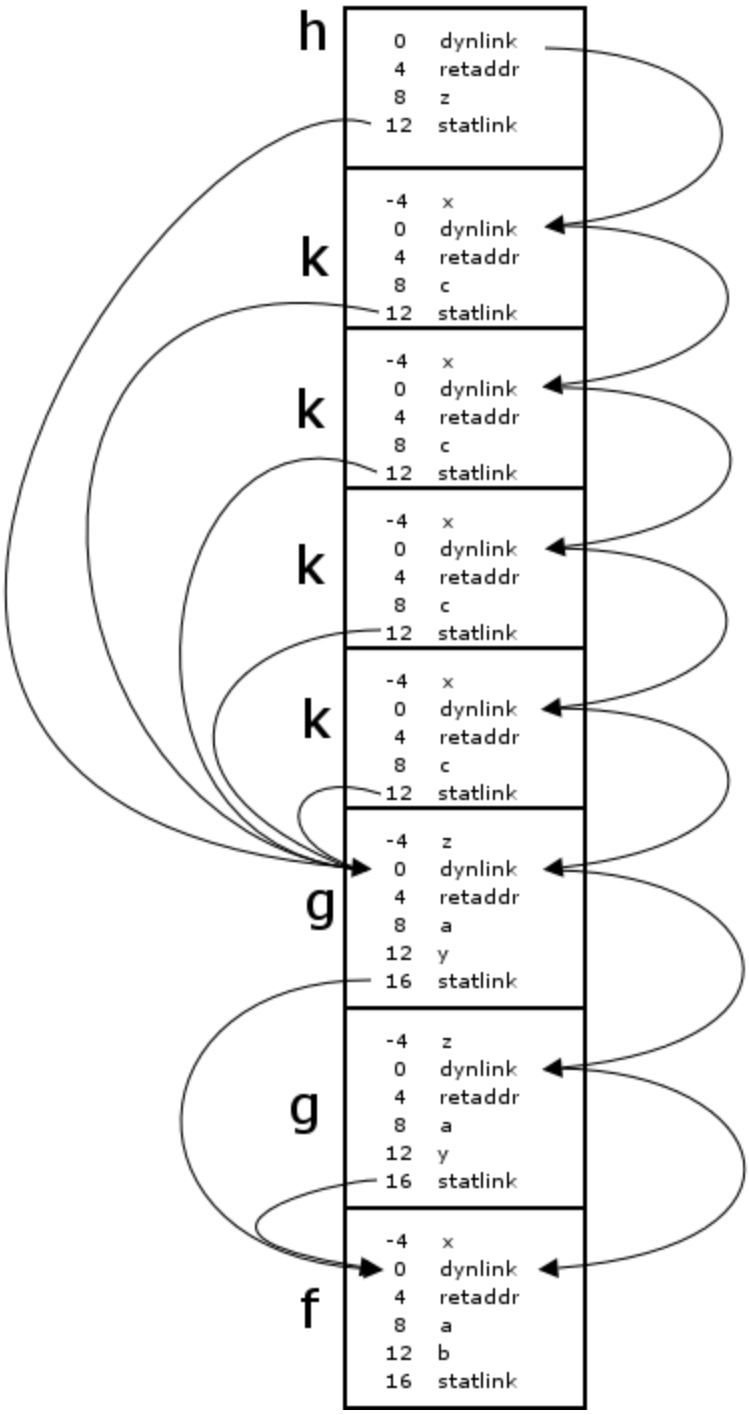
    g(0, 0);
}
```

The call sequence goes f → g → g → k → k → k → k → h. What gets printed? It depends on whether we have static or dynamic scoping.

If dynamically scoped, use a-lists or central reference tables.

If statically scoped, use *static links* (or *displays*). In this case we rely on the compiler to generate code that says:

> "To find x from the body of k, take two steps down the static chain and look in offset -4".



Heap Storage

TODO

Garbaage Collection

TODO

Process Management

TODO

Thread Management

TODO

Summary

We've covered:

- ✅ -
- ✅ -